Fondamenti di Programmazione

Spark

settembre e ottobre 2026 · volume 1

Capire un programma, un passo alla volta

Un programma cambia dei dati. Per capire se li cambia nel modo giusto dobbiamo prima dire che cosa vogliamo ottenere, poi seguire le operazioni e infine trovare un ragionamento che valga per tutti i dati ammessi. Questo libro costruisce con pazienza questi tre passaggi. Richiede soltanto aritmetica e algebra elementari; introduce il linguaggio logico quando diventa utile.

Il percorso ha cinque pilastri. **Specifica e stato** distinguono il risultato richiesto dalla procedura per ottenerlo (capitolo 1). Il **calcolo proposizionale** ci insegna a leggere condizioni e conseguenze (capitolo 2). La **logica del primo ordine** aggiunge oggetti, proprietà e quantificatori, indispensabili per parlare di tutti gli elementi di un array (capitoli 3 e 4). La **semantica dei comandi e la logica di Hoare** collegano le formule ai programmi (capitoli 5 e 6). Infine **invarianti, terminazione e zone degli array** permettono di verificare e costruire i cicli (capitoli 7–9).

Non occorre imparare subito ogni simbolo. Leggi prima l'esempio, chiediti che cosa vogliamo sapere, poi torna alla definizione. Negli esempi di esecuzione fermati prima di ogni assegnamento: quale valore viene letto? Quale viene sostituito? Nelle prove distingui ciò che sappiamo prima di un passo da ciò che dobbiamo sapere dopo.

Gli esercizi conclusivi hanno tutti una soluzione ragionata nell'ultima parte. Prova a scegliere il primo passo prima di leggerla. I passaggi «Per proseguire» sviluppano varianti secondarie; il filo principale resta comprensibile anche rimandandoli a una seconda lettura.

Il periodo va dalla prima nota disponibile, datata 19 settembre, al 31 ottobre 2026. L'ultima lezione presente nel corpus è del 29 ottobre. Il volume rielabora tredici note e la trascrizione del 25 settembre; le spiegazioni aggiunte non sono citazioni del docente. I documenti di accompagnamento registrano fonti, correzioni e selezione degli esempi.

Convenzioni. I naturali comprendono lo zero. I programmi sono scritti nel piccolo linguaggio imperativo del corso, con interi matematici: non sono ancora programmi C. L'assegnamento si scrive $x := E$ e cambia il valore di x; l'uguaglianza $x = E$ afferma una proprietà. Le variabili di specifica ricordano valori iniziali e non vengono modificate dal programma. La divisione intera e le condizioni di definitezza verranno precisate prima dell'uso.

Indice

Capire un programma, un passo alla volta . . . 2

1 Algoritmi, specifiche e stati . . . 5

1.1 Dal problema alla specifica . . . 5
1.2 Stato e assegnamento: che cosa cambia . . . 5
1.3 Algoritmi diversi, costi diversi . . . 6
1.4 Modelli di calcolo e limiti . . . 7

2 Calcolo proposizionale: verità e conseguenza . . . 8

2.1 Formule, connettivi e interpretazioni . . . 8
2.2 Modelli, tautologie e controesempi . . . 9
2.3 Equivalenze e dimostrazioni motivate . . . 9
2.4 Implicazione e polarità delle occorrenze . . . 10

3 Logica del primo ordine: individui e formule . . . 11

3.1 Termini, predicati e quantificatori . . . 11
3.2 Portata, variabili libere e sostituzione . . . 11
3.3 Interpretazioni: i simboli ricevono un significato . . . 12
3.4 Formalizzare: tutti, alcuni, esattamente uno . . . 12

4 Semantica e dimostrazioni con i quantificatori . . . 14

4.1 Assegnamenti e valore di termini e formule . . . 14
4.2 Negazione, distribuzione e domini ristretti . . . 15
4.3 Regole di prova: individui particolari e generici . . . 15
4.4 Uguaglianza e asserzioni sugli stati . . . 16

5 Stati, espressioni e comandi . . . 17

5.1 Lo stato e la valutazione delle espressioni . . . 17
5.2 Assegnamento, sequenza e perdita di informazione . . . 17
5.3 Scelte e ripetizioni: if e while . . . 18
5.4 La semantica dei comandi e le esecuzioni senza risultato . . . 19

6 Triple di Hoare e invarianti dei cicli . . . 20

6.1 Precondizione, postcondizione e correttezza . . . 20
6.2 Conseguenza e definitezza delle espressioni . . . 20
6.3 Assegnamento: ragionare a partire dal risultato . . . 21
6.4 Sequenza: trovare l'asserzione intermedia . . . 21
6.5 Condizionale: utilizzare l'informazione della guardia . . . 22
6.6 Invariante e correttezza parziale del while . . . 22

7 Correttezza totale e funzioni di terminazione . . . 24

7.1 Una quantità naturale che diminuisce . . . 24
7.2 La regola del while in correttezza totale . . . 24
7.3 Divisione intera: completare la prova . . . 25
7.4 Somma dei primi interi: scegliere l'invariante . . . 25
7.5 Programmi annotati e proprietà non modificate . . . 26

8 Costruire un ciclo: il significato delle variabili . . . 28

8.1 Dalla postcondizione a una proprietà da conservare . . . 28
8.2 Fattoriale: ricordare il prodotto già calcolato . . . 28

8.3 MCD: trasformare i dati conservando la risposta 29
8.4 Radice cubica: costruire potenze con differenze 29
8.5 Fibonacci: conservare il valore che manca 30

9 Array: indici, aggiornamenti e zone 32

9.1 Array come funzioni sugli indici 32
9.2 Aggiornamento selettivo: prima valutare, poi modificare 32
9.3 Incrementare un array: un confine tra fatto e da fare 33
9.4 Somma di un array: sostituire la fine con un confine mobile 34
9.5 Bandiera a due colori: classificare senza perdere elementi 34

Appendice A Soluzioni ragionate 37

A.1 Capitolo 1 — Specifiche e stati 37
A.2 Capitolo 2 — Calcolo proposizionale 37
A.3 Capitolo 3 — Individui e formule 37
A.4 Capitolo 4 — Semantica e quantificatori 37
A.5 Capitolo 5 — Stati, espressioni e comandi 38
A.6 Capitolo 6 — Triple di Hoare e invarianti 38
A.7 Capitolo 7 — Correttezza totale 38
A.8 Capitolo 8 — Costruire un ciclo 38
A.9 Capitolo 9 — Array 39

Capitolo 1

Algoritmi, specifiche e stati

Vogliamo calcolare $5 \cdot 3$. Una calcolatrice può moltiplicare direttamente. Un esecutore che conosce soltanto somme, sottrazioni e confronti deve fare altro: sommare 5 tre volte. Il risultato richiesto è lo stesso, ma il procedimento dipende dalle operazioni disponibili. Da questa distinzione nasce il primo pilastro del libro: separare *che cosa* vogliamo ottenere da *come* lo otteniamo.

1.1 Dal problema alla specifica

«Trova il numero di telefono di una persona» sembra una richiesta chiara. Ma che cosa accade se due persone hanno lo stesso nome? E se il nome non compare? Per programmare dobbiamo sciogliere queste ambiguità prima di scegliere i passi del calcolo.

Una **specifica** descrive quali dati sono ammessi all'inizio e quale relazione deve esserci fra questi dati e il risultato. Per il prodotto: dati due interi non negativi a, b, ottenere $a\,b$. Abbiamo esteso l'esempio positivo delle lezioni al caso zero, che potremo controllare esplicitamente. La specifica non impone ancora somme, moltiplicazioni o altri strumenti.

Il **pensiero computazionale** aiuta a precisarla. Facciamo *astrazione* quando conserviamo soltanto i dati rilevanti; *scomposizione* quando dividiamo il compito in parti; riconosciamo *schemi* quando due problemi ammettono la stessa struttura di soluzione. Costruiamo infine una procedura abbastanza precisa da poter essere automatizzata. L'informatica studia questi algoritmi, la loro progettazione e analisi, le loro possibilità e i loro limiti.

> **Concetto chiave · Algoritmo e programma**
> Un algoritmo descrive un procedimento mediante regole finite, non ambigue ed eseguibili da un esecutore adatto. Un programma lo esprime in un linguaggio di programmazione. Per un problema che richiede un risultato, chiediamo anche che l'esecuzione termini su tutti i dati ammessi e produca quel risultato.

Una descrizione finita può contenere una ripetizione: «finché rimangono somme, eseguine una» è una sola regola, anche se viene usata molte volte. La finitezza della descrizione e la terminazione dell'esecuzione sono quindi due questioni distinte. «Aggiungi sale quanto basta» lascia invece indeterminato il controllo da eseguire: per un esecutore automatico occorre una condizione precisa.

Il lavoro si articola in specifica, progetto dell'algoritmo, codifica ed esecuzione. Se qualcosa non funziona, il **debugging** cerca dove nasce l'errore: nella richiesta, nel procedimento, nella traduzione o nei singoli passi. Un debugger aiuta a osservare l'esecuzione; un test riuscito ci informa sul caso provato. Per garantire tutti i casi ammessi servirà una dimostrazione.

1.2 Stato e assegnamento: che cosa cambia

Nel prodotto per somme ci servono tre informazioni: il valore da aggiungere, quante aggiunte restano e quanto abbiamo già accumulato. Chiamiamole A, B, C. Uno stato raccoglie i loro valori in un certo momento:

$$\sigma = \{A \mapsto 5, B \mapsto 3, C \mapsto 0\}.$$

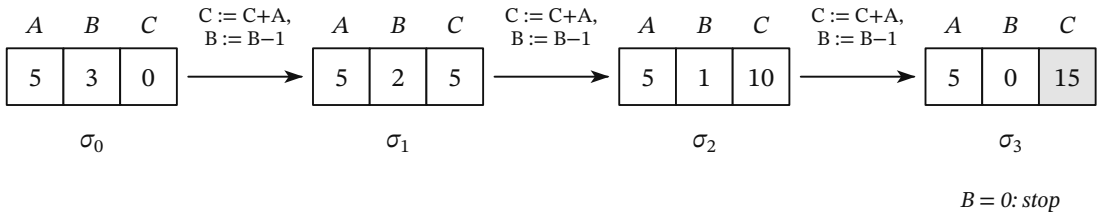
La scrittura $B \mapsto 3$ significa che il nome B è associato al valore 3. Il nome ci permette di ritrovare l'informazione; il valore può cambiare. Lo stato è una funzione parziale dai nomi ai valori: a ogni nome associa al più un valore, mentre alcuni nomi possono non comparire. $\{x \mapsto 3, y \mapsto 3\}$ è ammesso, perché nomi diversi possono avere lo stesso valore; $\{x \mapsto 3, x \mapsto 4\}$ non lo è, perché la lettura di x sarebbe ambigua.

L'**assegnamento** $C := C + A$ legge i valori correnti di C e A, ne calcola la somma e sostituisce il valore di C. Nello stato precedente ottiene $C = 5$. Non è un'equazione da risolvere: i due usi di C riguardano due momenti diversi. Il valore di A resta 5 e quello di B resta 3.

Possiamo dunque scrivere il procedimento:

```
C := 0;
while B > 0 do
  C := C + A;
  B := B - 1
endw
```

La guardia $B > 0$ viene controllata prima di ogni giro. Se è vera, si eseguono nell'ordine le due istruzioni; poi si torna al controllo. Se è falsa, si esce senza eseguire il corpo. Con $B = 0$ all'inizio il risultato resta 0: il caso zero è già compreso.



Stati ai controlli della guardia per $A = 5, B = 3$. Tra due colonne si eseguono prima l'aggiunta e poi il decremento.

Perché diminuire B? Per ricordare quante aggiunte mancano. Perché iniziare C da zero? Perché prima di ogni aggiunta non abbiamo ancora accumulato nulla. Dopo il primo giro abbiamo fatto una somma e ne restano due; dopo il secondo abbiamo 10 e ne resta una; dopo il terzo abbiamo 15 e non ne resta nessuna.

Per controllare tutti i casi, vorremmo conservare il valore iniziale del contatore. Lo chiamiamo b; analogamente chiamiamo a il valore iniziale di A. Questi sono **valori di specifica**, non caselle aggiornate dal programma. A ogni controllo la quantità già prodotta, C, più quella ancora da produrre, $A\,B$, vale $a\,b$. Questa osservazione anticipa il metodo degli *invarianti*, che costruiremo nel capitolo 6. All'inizio vale $0 + a\,b = a\,b$. In un giro aggiungiamo A a C e togliamo 1 a B: la parte accumulata cresce esattamente di quanto cala quella ancora da produrre. In simboli, $(C + A) + A(B - 1) = C + A\,B$. Quando $B = 0$, resta $C = a\,b$. Inoltre B è un naturale che diminuisce di 1 a ogni giro, quindi quel momento viene raggiunto.

Non basta scrivere alla fine «$C = A\,B$»: nello stato finale $B = 0$, quindi quel prodotto non ricorda più l'ingresso. La richiesta corretta è $C = a\,b$. Se vogliamo preservare anche gli ingressi, dobbiamo dichiararlo nella specifica e usare un contatore ausiliario.

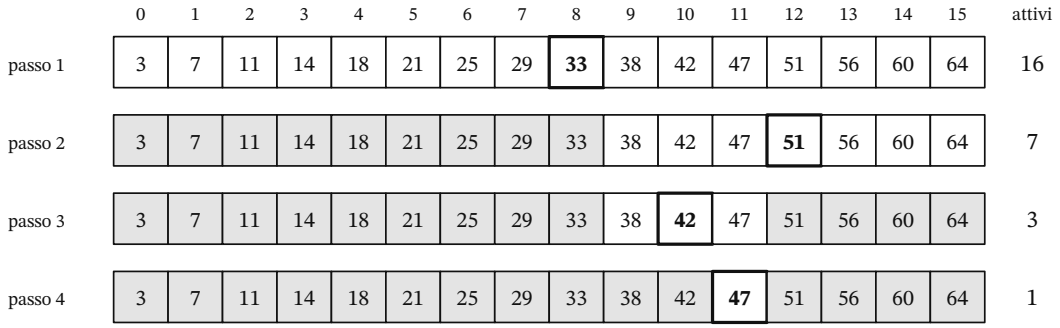
1.3 Algoritmi diversi, costi diversi

Il prodotto per somme impiega b giri. Possiamo sfruttare una regolarità per ridurli. Se il primo fattore è pari, dimezzarlo e raddoppiare il secondo conserva il prodotto. Se è dispari e maggiore di 1, togliamo un'unità e mettiamo da parte una copia del secondo fattore. Con $13 \cdot 11$ otteniamo:

$$13 \cdot 11 = 12 \cdot 11 + 11 = 6 \cdot 22 + 11 = 3 \cdot 44 + 11.$$

Ora $3 \cdot 44 = 2 \cdot 44 + 44 = 1 \cdot 88 + 44$, quindi il risultato è $88 + 44 + 11 = 143$. È l'idea della moltiplicazione egizia richiamata nella lezione introduttiva: un procedimento antico mette in luce come usare proprietà dei numeri per progettare un algoritmo.

Un secondo esempio è la ricerca in un elenco ordinato. Confrontiamo il nome cercato con quello centrale. Se il cercato viene prima, tutti i nomi successivi possono essere esclusi; se viene dopo, escludiamo tutti i precedenti. L'ordine giustifica lo scarto. Senza ordinamento, eliminare una metà non sarebbe lecito.



Ricerca di 47: i valori confrontati sono 33, 51, 42, 47. La zona attiva si restringe perché i valori sono ordinati.

Qui contiamo un confronto a tre esiti con l'elemento centrale come un passo. Il numero dei candidati si riduce circa della metà, mentre in una ricerca sequenziale si esaminano uno dopo l'altro. Dire che un algoritmo è corretto non dice ancora quanto costa eseguirlo: **correttezza** ed **efficienza** sono proprietà diverse.

1.4 Modelli di calcolo e limiti

Possiamo rendere precisa l'idea di esecutore senza costruire un calcolatore fisico. Una **macchina di Turing** usa un nastro di simboli, una testina e regole finite; può leggere, scrivere e cambiare posizione e stato di controllo. La macchina universale riceve la descrizione di un'altra macchina e il suo ingresso e ne simula il comportamento: uno stesso esecutore può svolgere programmi diversi.

Il **λ-calcolo** di Church descrive invece il calcolo attraverso funzioni e applicazioni. Pur avendo forme diverse, i due formalismi calcolano le stesse funzioni. La **tesi di Church–Turing** identifica la nozione intuitiva di procedimento effettivo con quella formalizzata da questi modelli. È una tesi, perché uno dei due termini del confronto è intuitivo; l'equivalenza fra i modelli formali è invece un risultato matematico.

L'architettura di **von Neumann** conserva dati e istruzioni in memoria. Quando ne discutiamo la capacità astratta di calcolo assumiamo memoria estendibile: un dispositivo reale ha risorse finite. Altri modelli citati nel corso, come sistemi di Post e funzioni ricorsive, illuminano lo stesso concetto; modelli quantistici e biologici vengono qui richiamati soltanto come prospettive sul calcolo, senza confrontarne l'efficienza.

Non ogni richiesta precisa ammette un algoritmo generale. Il **problema dell'arresto** chiede di decidere, per ogni programma e ogni ingresso, se l'esecuzione terminerà. È indecidibile: non esiste un algoritmo che dia sempre la risposta corretta e termini su tutti questi casi. Ciò non impedisce di dimostrare la terminazione di specifici programmi, come faremo nei capitoli 7 e 8.

Un linguaggio può organizzare il calcolo mediante comandi che cambiano lo stato (*paradigma imperativo*, come C), funzioni (*funzionale*), oggetti (*a oggetti*), relazioni e inferenze (*logico*, come Prolog), oppure processi che interagiscono (*concorrente*). Il nostro percorso si concentra sulla programmazione imperativa. Prima di introdurre altri comandi, dobbiamo imparare a esprimere le proprietà degli stati: è il compito della logica.

Verifica 1

1. Nell'algoritmo del prodotto, parti da $A = 4, B = 2$. Scrivi gli stati ai controlli della guardia, spiega il ruolo di C e indica perché $C = A\,B$ non è la richiesta finale corretta.

2. Vuoi restituire il maggiore fra due interi, conservando gli ingressi. Scrivi una specifica che distingua ingressi, risultato e vincoli. Spiega perché la richiesta non stabilisce ancora l'algoritmo.

Calcolo proposizionale: verità e conseguenza

«Se Corrado va al cinema, ci va Antonio; se Antonio va al cinema, ci va Bruno.» Possiamo concludere che, se ci va Corrado, ci va Bruno? Sì: quando Corrado va, la prima premessa porta ad Antonio e la seconda a Bruno. Non possiamo invece concludere che, se va Bruno, vada Corrado. Bruno potrebbe andare da solo. Il calcolo proposizionale rende preciso questo controllo: una conclusione segue se non possiamo avere premesse vere e conclusione falsa.

2.1 Formule, connettivi e interpretazioni

Una **proposizione** è una frase dichiarativa alla quale attribuiamo vero o falso. Una domanda e un comando non sono proposizioni. «$x + 1 = 2$» diventerà vera o falsa quando avremo fissato x; torneremo su questa dipendenza con il primo ordine.

Per ora rappresentiamo le proposizioni semplici con lettere $p, q, r, \ldots$. Da queste costruiamo formule: $\neg P$ significa «non P», $P \wedge Q$ «P e Q», $P \vee Q$ «almeno una fra P e Q». La disgiunzione è inclusiva: ammette che entrambe siano vere. $P \Rightarrow Q$ significa «se P, allora Q»; $P \equiv Q$ «P se e solo se Q».

Il significato di «se» merita una sosta. «Se piove, porto l'ombrello» esclude il solo caso in cui piove e non porto l'ombrello. Quando non piove, l'affermazione non impone nulla sull'ombrello. Perciò $P \Rightarrow Q$ è falsa soltanto con P vera e Q falsa. L'equivalenza, invece, è vera quando i due valori coincidono.

Indichiamo i valori con t e f. Questa tabella definisce le operazioni booleane:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \equiv Q$
t	t	f	t	t	t	t
t	f	f	f	t	f	f
f	t	t	f	t	t	f
f	f	t	f	f	t	t

La **sintassi** specifica quali stringhe sono formule. Fissato un alfabeto di lettere proposizionali, sono formule le lettere, le costanti T, F e, ricorsivamente, le espressioni ottenute applicando i connettivi a formule già costruite. T indica la formula sempre vera, F quella sempre falsa: non sono lettere con un valore da scegliere. Una grammatica libera dal contesto raccoglie queste regole di costruzione; «ricorsivamente» significa che possiamo applicarle ancora ai risultati già ottenuti. Stringhe come $p \wedge \vee$ non si ottengono così.

Le parentesi descrivono la costruzione. Nel corso $\neg$ lega più di $\wedge, \vee$, che legano più di $\Rightarrow$, che lega più di $\equiv$. Fra $\wedge$ e $\vee$ non scegliamo una precedenza: scriviamo le parentesi. Anche due implicazioni consecutive richiedono parentesi. Con P, Q false, $(P \Rightarrow \neg P) \Rightarrow Q$ è falsa, mentre $P \Rightarrow (\neg P \Rightarrow Q)$ è vera. Le parentesi hanno cambiato la domanda logica.

Un'**interpretazione** I assegna a ogni lettera un valore booleano. La semantica valuta poi la formula seguendone la costruzione. Con $I(p) = \mathrm{t}$, $I(q) = I(r) = \mathrm{f}$, per valutare $(p \wedge q) \vee \neg r$ calcoliamo prima $p \wedge q$, che è falsa, e $\neg r$, che è vera; la loro disgiunzione è vera. Scriviamo $[\![P]\!]_I$ per il valore di P in I.

Per una **tavola di verità** ripetiamo la valutazione per tutte le scelte dei valori. Con n lettere distinte ci sono 2^n scelte, perché ogni lettera ha due possibilità indipendenti.

2.2 Modelli, tautologie e controesempi

Un'interpretazione che rende vera P è un suo **modello**; scriviamo $I \vDash P$. Una formula è **soddisfacibile** se ha almeno un modello; è una **tautologia** se ogni interpretazione è un modello; è una **contraddizione** se non ne ha. $p \Rightarrow q$ è soddisfacibile, perché è vera con p, q vere, ma non è una tautologia: p vera e q falsa la falsificano. Invece $p \vee \neg p$ è una tautologia e $p \wedge \neg p$ una contraddizione.

Concetto chiave · Conseguenza logica
$\Gamma \vDash Q$ significa che ogni interpretazione che rende vere tutte le premesse di Γ rende vera anche Q. Per confutare la conseguenza basta un **controesempio**: premesse tutte vere e conclusione falsa.

Riprendiamo gli amici: $c \Rightarrow a$ e $a \Rightarrow b$. La conclusione $c \Rightarrow b$ segue: se fosse falsa avremmo c vera e b falsa. La prima premessa imporrebbe a vera, e la seconda sarebbe allora falsa. Non esiste quindi un controesempio. Per $b \Rightarrow c$, invece, scegliamo b vera, a, c false: entrambe le premesse sono vere, la conclusione no.

Non concludiamo neppure che nessuno va al cinema: l'interpretazione con tutti e tre veri soddisfa le premesse. Una condizione necessaria non è automaticamente sufficiente. «Antonio va *solo se* va Bruno» si scrive $a \Rightarrow b$: Antonio che va obbliga Bruno, non viceversa.

Con premesse finite $P_1, \ldots, P_n$, la conseguenza equivale alla tautologicità di $(P_1 \wedge \cdots \wedge P_n) \Rightarrow Q$. Infatti la formula è falsa esattamente nei controesempi. Questo permette di controllare le inferenze con una tavola di verità, ma il numero delle righe cresce rapidamente. Le leggi ci danno un altro modo di ragionare.

2.3 Equivalenze e dimostrazioni motivate

Una **legge** è una tautologia che usiamo per giustificare passaggi. Se $P \equiv Q$ è una legge, possiamo sostituire una sottoformula P con Q dentro una formula più grande: i valori restano uguali in ogni interpretazione. È il principio di sostituzione per l'equivalenza. Per esempio $P \Rightarrow Q$ si può sostituire con $\neg P \vee Q$, perché entrambe sono false nel solo caso P vera, Q falsa.

Alcune leggi esprimono operazioni familiari: commutatività e associatività di $\wedge, \vee$, idempotenza $P \wedge P \equiv P$ e $P \vee P \equiv P$. Le unità sono $P \wedge T \equiv P$ e $P \vee F \equiv P$; gli elementi assorbenti danno $P \wedge F \equiv F$ e $P \vee T \equiv T$. Le leggi distributive consentono di espandere $P \wedge (Q \vee R)$ in $(P \wedge Q) \vee (P \wedge R)$ e, scambiando $\wedge$ con $\vee$, di espandere $P \vee (Q \wedge R)$.

La doppia negazione dà $\neg\neg P \equiv P$. Le leggi di De Morgan dicono

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q, \qquad \neg(P \vee Q) \equiv \neg P \wedge \neg Q.$$

Per capire la prima, chiediamoci quando fallisce «entrambi»: basta che fallisca almeno uno. La seconda dice che per fallire «almeno uno» devono fallire entrambi. Inoltre $\neg(P \Rightarrow Q) \equiv P \wedge \neg Q$, ed $P \equiv Q$ equivale sia a $(P \Rightarrow Q) \wedge (Q \Rightarrow P)$ sia a $(P \wedge Q) \vee (\neg P \wedge \neg Q)$.

Vediamo perché può essere utile distribuire, anziché farlo senza una meta. Vogliamo semplificare $P \vee (\neg P \wedge Q)$. Qui P compare anche negata: distribuendo portiamo le due occorrenze vicine:

$$P \vee (\neg P \wedge Q) \equiv (P \vee \neg P) \wedge (P \vee Q) \equiv T \wedge (P \vee Q) \equiv P \vee Q.$$

Il primo passo è distributività; il secondo usa il terzo escluso; l'ultimo usa l'unità. Questa è una legge di complemento. Anche $P \wedge (\neg P \vee Q) \equiv P \wedge Q$; le leggi di assorbimento sono $P \wedge (P \vee Q) \equiv P$ e $P \vee (P \wedge Q) \equiv P$. Non occorre memorizzarle tutte: occorre capire quale cambiamento permette ogni passaggio.

Per dimostrare un'implicazione possiamo assumere la premessa e ricavare la conclusione. Da $P \Rightarrow Q$ e P segue Q: è il **modus ponens**, perché la sola possibilità di Q falsa renderebbe falsa una premessa. Da $P \Rightarrow Q$ e $\neg Q$ segue $\neg P$ (**modus tollens**): se P fosse vera, costringerebbe Q a esserlo. Qui abbiamo usato un ragionamento per assurdo, non una nuova operazione misteriosa.

La **controposizione** esprime proprio $(P \Rightarrow Q) \equiv (\neg Q \Rightarrow \neg P)$. Non scambia semplicemente i due lati: li nega entrambi. La prova per assurdo di $P \Rightarrow Q$ cerca una contraddizione assumendo $P \wedge \neg Q$, l'unico caso che potrebbe falsificare l'implicazione. Una prova per casi sceglie una proposizione R e dimostra la stessa conclusione sia con R sia con $\neg R$; questi due casi esauriscono le possibilità.

2.4 Implicazione e polarità delle occorrenze

L'equivalenza consente sostituzioni in ogni posizione; una sola implicazione richiede attenzione. Da $A \Rightarrow B$ sappiamo che A è più esigente di B. Perciò $A \wedge R \Rightarrow B \wedge R$. Ma sotto negazione il verso cambia: $\neg B \Rightarrow \neg A$. Se nessuno soddisfa la richiesta più debole B, tanto meno soddisfa quella più forte A.

Per seguire questi cambiamenti chiamiamo un'occorrenza **positiva** se dalla radice alla sottoformula attraversiamo un numero pari di inversioni, **negativa** se dispari. Invertono la negazione e l'antecedente di $\Rightarrow$; $\wedge, \vee$ e il conseguente non invertono. Prima eliminiamo gli eventuali $\equiv$, che non hanno questa monotonia. Conta ogni singola occorrenza, non il nome della lettera.

Se vale $A \Rightarrow B$, sostituire A con B in posizione positiva dà un'implicazione dalla formula iniziale alla nuova; in posizione negativa dà il verso opposto. Per esempio, da $Q \wedge R \Rightarrow Q$ segue $(P \Rightarrow (Q \wedge R)) \Rightarrow (P \Rightarrow Q)$: il conseguente è positivo. Da $P \Rightarrow P \vee Q$ segue invece $((P \vee Q) \Rightarrow R) \Rightarrow (P \Rightarrow R)$: allargare la premessa rende più esigente l'implicazione.

Le ipotesi usate rimangono parte del ragionamento. Se abbiamo assunto H, il risultato dipende da H; non diventa una legge assoluta. Per esempio, sotto l'ipotesi P, possiamo scrivere $P \wedge Q \equiv T \wedge Q \equiv Q$. Abbiamo dimostrato $P \Rightarrow ((P \wedge Q) \equiv Q)$, non $(P \wedge Q) \equiv Q$ senza ipotesi.

2.4.1 Per proseguire: forme normali, risoluzione e NAND

Un **letterale** è una lettera o la sua negazione. Una **forma normale congiuntiva** (FNC) è una congiunzione di disgiunzioni di letterali; una **forma normale disgiuntiva** (FND) scambia i ruoli. Per esempio $(p \vee \neg q) \wedge r$ è in FNC; $(p \wedge q) \vee \neg r$ in FND. Si eliminano $\Rightarrow, \equiv$, si spingono le negazioni verso le lettere con De Morgan e si distribuisce. Il procedimento può allungare molto le formule; una forma normale non è una forma unica.

La **risoluzione** usa due clausole $P \vee Q$ e $\neg P \vee R$ per ricavare $Q \vee R$. Perché? Se P è falsa, la prima clausola impone Q; se è vera, la seconda impone R. È una prova per casi. Dalle tre clausole $P \vee Q$, $\neg P \vee R$, $\neg Q \vee R$ ricaviamo prima $Q \vee R$, poi, risolvendo su Q, R. La risoluzione è uno strumento dei metodi automatici e della programmazione logica.

Un insieme di connettivi è **funzionalmente completo** se esprime ogni funzione booleana. $\{\neg, \wedge\}$ lo è: De Morgan esprime $\vee$, e la FND esprime una funzione scegliendo le righe della sua tavola in cui è vera. Anche $\{\neg, \vee\}$ è completo per De Morgan, e $\{\neg, \Rightarrow\}$ lo è perché $P \vee Q \equiv (\neg P \Rightarrow Q)$. Il solo **NAND**, $P \uparrow Q \equiv \neg(P \wedge Q)$, basta: $P \uparrow P \equiv \neg P$ e $(P \uparrow Q) \uparrow (P \uparrow Q) \equiv P \wedge Q$. Più connettivi rendono comunque più leggibile ciò che vogliamo dire.

VERIFICA 2

1. Con p: «piove» ed e: «esco», traduci «esco solo se non piove». La formula ottenuta implica «se non piove, esco»? Cerca un caso che distingua le due richieste.

2. Semplifica $P \wedge (\neg P \vee Q)$ mostrando le leggi usate. Poi stabilisci il segno delle occorrenze di P, Q, R in $\neg(P \Rightarrow Q) \wedge R$.

Logica del primo ordine: individui e formule

«Tutti i numeri pari hanno successore dispari.» Potremmo introdurre una lettera per il caso di 0, una per il caso di 2, e così via, ma non avremmo scritto la regola generale. Ci serve vedere dentro gli atomi: il numero, la proprietà «pari», l'operazione «successore». La **logica del primo ordine** aggiunge questi ingredienti e permette di dire «per ogni» ed «esiste».

3.1 Termini, predicati e quantificatori

Scegliamo prima gli oggetti di cui parlare, il **dominio**. Per l'esempio usiamo $\mathbb{N} = \{0, 1, 2, \dots\}$. Introduciamo la costante 1, la funzione somma, i predicati pari e dispari, e una variabile x. La scrittura

$$\forall x.(\text{pari}(x) \Rightarrow \text{dispari}(x+1))$$

dice che, qualunque numero sia scelto per x, se è pari il suo successore è dispari. Il simbolo $\forall$ è il **quantificatore universale**. Per dire che almeno un numero è pari usiamo l'**esistenziale** $\exists x.\text{pari}(x)$.

Un linguaggio del primo ordine comprende **costanti**, **variabili**, **simboli di funzione** e **simboli di predicato**, oltre a connettivi e quantificatori. Ogni funzione e ogni predicato ha un'**arietà**: quanti argomenti richiede. Il successore ha arietà 1, la somma arietà 2, il predicato «minore» arietà 2. Un predicato di arietà zero si comporta come una lettera proposizionale.

I **termini** sono le scritture che denotano individui: ogni costante e ogni variabile è un termine; se f ha arietà n e $t_1, \dots, t_n$ sono termini, anche $f(t_1, \dots, t_n)$ è un termine. Per esempio, con s successore, $s(s(x))$ è un termine costruito applicando due volte la stessa regola. Non è un'affermazione: non è vero o falso, indica un numero.

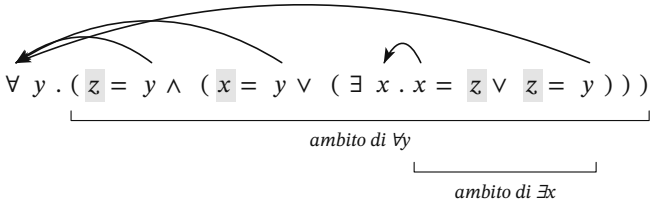
Le **formule atomiche** applicano un predicato a termini: $\text{pari}(s(s(x)))$ oppure $x < y$. Da queste, da T, F, e dai connettivi costruiamo formule come prima. La nuova regola consente, data una formula P, di costruire $\forall x.P$ ed $\exists x.P$. Quantificatori consecutivi uguali si possono abbreviare, per esempio $\forall x, y.P$.

> **Concetto chiave · Termine e formula**
> Un termine denota un individuo; una formula esprime una condizione vera o falsa. Le funzioni prendono individui e restituiscono individui; i predicati prendono individui e restituiscono valori booleani. $x + 1$ è un termine, $\text{pari}(x + 1)$ una formula. $\text{pari}(x) + 1$ non rispetta questi ruoli.

3.2 Portata, variabili libere e sostituzione

In $\forall x.P$ la formula P è la **portata** del quantificatore. Un'occorrenza di x è **legata** quando si trova nella portata di un quantificatore su x; altrimenti è **libera**. Se vi sono quantificatori annidati con lo stesso nome, conta quello più interno che comprende l'occorrenza.

In $(\forall x.p(x)) \wedge q(x, y)$ la prima x è legata; la seconda x e y sono libere. Dunque una stessa lettera può avere occorrenze con ruoli diversi. Una formula è **chiusa** se non ha variabili libere, **aperta** altrimenti. La formula $x + 1 = 2$ dipende dal valore esterno di x; $\exists x.x + 1 = 2$ afferma invece l'esistenza di un valore adatto.



Le frecce identificano il quantificatore che lega ogni occorrenza. Nella formula illustrata tutte le y sono legate, tutte le z libere; la x esterna a $\exists x$ rimane libera.

La notazione $P[t/x]$ significa: sostituisci il termine t alle occorrenze *libere* di x in P. Non cambiamo le occorrenze già governate da un quantificatore interno su x. Inoltre nessuna variabile libera del termine inserito deve diventare legata per effetto della sostituzione: sarebbe una **cattura**.

Vediamo un caso che spiega la precauzione. In $P = \exists y.x < y$ vogliamo sostituire y a x. Scrivere $\exists y.y < y$ sarebbe sbagliato: la y inserita, che doveva restare libera, è stata catturata. Prima rinominiamo il segnaposto legato: $\exists z.x < z$. Ora sostituiamo e otteniamo $\exists z.y < z$. Sui naturali questa formula dice correttamente che il numero y ha un numero più grande.

La **ridenominazione delle variabili legate** non cambia il significato se usa un nome nuovo: $\forall x.P$ equivale a $\forall z.P[z/x]$, purché z non compaia in P, sostituendo solo le occorrenze di x libere nella portata. Il nome è un segnaposto; ciò che conta è quali occorrenze sono collegate a quel quantificatore.

3.3 Interpretazioni: i simboli ricevono un significato

La formula $\forall x.p(x)$ non dice ancora quali oggetti abbiano quale proprietà. Un'**interpretazione** $I = (D, \alpha)$ specifica un dominio non vuoto D e il significato dei simboli: ogni costante denota un elemento di D; ogni funzione di arietà n denota una funzione totale $D^n \to D$; ogni predicato di arietà n denota una funzione $D^n \to \mathrm{Bool}$. D^n è l'insieme delle liste ordinate di n elementi di D. Per l'uguaglianza richiederemo il significato usuale di identità.

Il dominio conta anche per gli elementi senza nome. Se $D = \{5, 10, 15\}$ e p significa «essere multiplo di 5», $\forall x.p(x)$ è vera. Se ampliamo il dominio a $\mathbb{N}$ e manteniamo lo stesso significato, la formula è falsa: 7 è un controesempio. Avere costanti che nominano soltanto 5, 10 e 15 non elimina gli altri numeri dal dominio dei quantificatori.

Costanti diverse possono denotare lo stesso individuo, se non imponiamo che siano distinti. Simboli dall'aspetto familiare, come $+$, non hanno automaticamente l'interpretazione aritmetica: occorre dichiararla. Nel seguito, quando lavoriamo sui numeri e sui programmi, useremo esplicitamente quella usuale.

3.4 Formalizzare: tutti, alcuni, esattamente uno

Prendiamo come dominio le persone e come predicati $S(x)$: «x segue FPL», $C(x)$: «x va al cinema». «Tutti gli studenti di FPL vanno al cinema» diventa $\forall x.(S(x) \Rightarrow C(x))$. Una persona che non segue FPL non è un controesempio: l'enunciato non la riguarda. «Qualche studente di FPL va al cinema» diventa invece $\exists x.(S(x) \wedge C(x))$: il testimone deve soddisfare entrambe le condizioni.

Questo spiega le abbreviazioni della **quantificazione ristretta**:

$$(\forall x\colon S(x).C(x)) \equiv (\forall x.(S(x) \Rightarrow C(x))),$$

$$(\exists x\colon S(x).C(x)) \equiv (\exists x.(S(x) \wedge C(x))).$$

Usare un'implicazione nell'esistenziale darebbe un significato diverso: basterebbe una persona che non segue FPL, anche se nessuno studente andasse al cinema. Usare una congiunzione nell'universale imporrebbe invece che tutte le persone seguano FPL.

«Tutti tranne uno» richiede sia un'eccezione sia la sua unicità. Costruiamo prima la proprietà dell'eccezione, $E(x) = S(x) \wedge \neg C(x)$. Dire «esattamente uno studente di FPL non va al cinema» significa

$$\exists x.(E(x) \wedge \forall y.(E(y) \Rightarrow y = x)).$$

La prima parte dà un'eccezione; la seconda impone che ogni eccezione coincida con essa. Equivalentemente possiamo dire che tutti gli altri studenti vanno al cinema. Senza la seconda parte avremmo detto soltanto «almeno uno».

Il dominio e il vocabolario si scelgono in funzione della domanda. Per «due persone sono parenti se hanno un antenato comune» servono predicati binari $A(z, x)$: «z è antenato di x» e $R(x, y)$: «sono parenti». L'antenato senza nome diventa una variabile:

$$\forall x, y.((\exists z.(A(z, x) \wedge A(z, y))) \Rightarrow R(x, y)).$$

«Se» dà questa direzione; «se e solo se» ne imporrebbe anche un'altra. La formalizzazione richiede dunque interpretare il senso della frase, poi controllare che la formula dica proprio quello.

Verifica 3

1. Nella formula $(\forall x.p(x, y)) \wedge (\exists y.q(x, y))$ individua le occorrenze libere e legate. Sostituisci poi y alle occorrenze libere di x, evitando catture.

2. Sui naturali formalizza «ogni numero pari ha il successore del successore pari», usando la funzione s di successore. Formalizza poi «esattamente un elemento soddisfa p», spiegando il compito dell'uguaglianza.

Semantica e dimostrazioni con i quantificatori

Per valutare $x+1=4$ dobbiamo sapere che numero rappresenta x. Per valutare $\exists x.x+1=4$ dobbiamo invece provare se qualche numero rende vera la condizione. La stessa idea, dare temporaneamente un valore a una variabile, ci permette di definire entrambe le valutazioni.

4.1 Assegnamenti e valore di termini e formule

Fissiamo $I=(D,\alpha)$. Un **assegnamento** ρ associa a ogni variabile un elemento di D. La scrittura $\rho[d/x]$ indica l'assegnamento uguale a ρ sulle altre variabili, ma che dà d a x. Con $\rho(x)=2$, $\rho(y)=10$, l'assegnamento $\rho[3/x]$ dà 3 a x e ancora 10 a y.

Il valore di un termine, $\alpha_\rho(t)$, si calcola dalla sua struttura. Una variabile vale $\rho(x)$, una costante vale $\alpha(c)$; per $f(t_1,\ldots,t_n)$ calcoliamo prima i termini e poi applichiamo la funzione $\alpha(f)$ ai risultati. Per esempio, se s è il successore sui naturali e $\rho(x)=2$, $s(s(x))$ vale 4: prima 2, poi 3, infine 4.

Per una formula atomica valutiamo i termini e applichiamo il predicato. Se p significa «essere pari», $p(s(s(x)))$ è vera con quell'assegnamento. I connettivi mantengono le tabelle del capitolo 2. Scriviamo $I_\rho(P)$ per il valore della formula. Per i quantificatori:

Concetto chiave · Semantica dei quantificatori
$I_\rho(\forall x.P)$ è vero se $I_{\rho[d/x]}(P)$ è vero per *ogni* $d\in D$. $I_\rho(\exists x.P)$ è vero se ciò accade per *almeno un* $d\in D$. Altrimenti il rispettivo valore è falso.

Su un dominio finito $\{a,b,c\}$, un universale si comporta come la congiunzione dei tre casi e un esistenziale come la loro disgiunzione. Un elemento che rende vera la portata di un esistenziale è un **testimone**. Un elemento che rende falsa quella di un universale è un controesempio.

Consideriamo $D=\{a,b\}$, con Q vera su entrambi gli elementi, e $P(x,y)$ vera soltanto quando $x=a$. Vogliamo valutare

$$\exists x.(Q(x)\wedge\forall y.P(x,y)).$$

Cerchiamo un elemento che soddisfi entrambe le richieste. Scegliamo a: $Q(a)$ è vera, e per l'universale interno controlliamo $P(a,a)$ e $P(a,b)$, entrambe vere. Dunque a è un testimone e la formula è vera. Con b la verifica fallirebbe; ciò non cancella il testimone già trovato.

$P(x,y)$	$y=a$	$y=b$
$x=a$	t	t
$x=b$	f	f

La riga di a è tutta vera: per l'universale interno serve tutta la riga, per l'esistenziale esterno ne basta una.

Una formula dipende soltanto dai valori assegnati alle sue variabili libere. Perciò una formula chiusa non dipende da ρ, e scriviamo semplicemente $I(P)$. Le nozioni di modello, soddisfacibilità e conseguenza logica si estendono come nel capitolo 2. Una formula chiusa è **valida** se è vera in tutte le interpretazioni. Per una formula aperta, dire che è valida significa richiedere la verità per ogni interpretazione e per ogni assegnamento alle variabili libere. Questo è il senso in cui useremo le leggi schematiche. Essere vera sui naturali con le operazioni usuali è meno esigente: $\forall x.x+1>x$ vi è vera, ma non per qualunque interpretazione dei simboli.

4.2 Negazione, distribuzione e domini ristretti

Negare «tutti soddisfano P» significa che almeno uno non la soddisfa; negare «almeno uno soddisfa P» significa che tutti la falliscono. Sono le leggi di De Morgan:

$$\neg(\forall x.P) \equiv \exists x.\neg P, \qquad \neg(\exists x.P) \equiv \forall x.\neg P.$$

Nella quantificazione ristretta manteniamo il filtro: $\neg(\forall x\colon R.P) \equiv \exists x\colon R.\neg P$. Non neghiamo R, perché stiamo cercando proprio fra gli elementi ammessi. Se nessun elemento soddisfa R, $\forall x\colon R.P$ è **vacuamente vera**: non ci sono controesempi ammessi. $\exists x\colon R.P$ è falsa: manca ogni testimone. In particolare, «tutti gli elementi dell'array vuoto sono positivi» non afferma che nell'array vi siano elementi.

Un universale distribuisce su $\wedge$: ogni individuo soddisfa entrambe le condizioni esattamente quando tutti soddisfano la prima e tutti la seconda. Dualmente l'esistenziale distribuisce su $\vee$:

$$(\forall x.(P \wedge Q)) \equiv ((\forall x.P) \wedge (\forall x.Q)),$$

$$(\exists x.(P \vee Q)) \equiv ((\exists x.P) \vee (\exists x.Q)).$$

Le versioni scambiate danno soltanto implicazioni: $(\exists x.(P \wedge Q)) \Rightarrow ((\exists x.P) \wedge (\exists x.Q))$, perché un testimone comune è anche testimone di ciascuna proprietà; il viceversa fallisce se le due proprietà hanno testimoni diversi. Analogamente $((\forall x.P) \vee (\forall x.Q)) \Rightarrow \forall x.(P \vee Q)$, ma «ogni intero è pari o dispari» non impone che tutti siano pari o tutti dispari.

Se x non è libera in Q, Q non cambia mentre varia x. Possiamo estrarla: per esempio $\forall x.(P \wedge Q) \equiv (\forall x.P) \wedge Q$ ed $\exists x.(P \vee Q) \equiv (\exists x.P) \vee Q$. Qui usiamo il dominio non vuoto. Anche $\forall x.Q \equiv Q$ ed $\exists x.Q \equiv Q$ richiedono questa ipotesi. Se si ammettessero domini vuoti, $\forall x.F$ sarebbe vera e $\exists x.T$ falsa: non potremmo togliere il quantificatore.

I quantificatori dello stesso tipo si scambiano: $\forall x.\forall y.P \equiv \forall y.\forall x.P$, e analogamente per due esistenziali. Scambiare tipi diversi cambia invece le dipendenze. Sui naturali $\forall x.\exists y.x < y$ è vera: dopo aver visto x, scegliamo $y = x + 1$. $\exists y.\forall x.x < y$ è falsa: un solo y dovrebbe superare anche sé stesso. Vale sempre il verso da un testimone comune a testimoni che possono dipendere da x:

$$(\exists y.\forall x.P(x, y)) \Rightarrow (\forall x.\exists y.P(x, y)).$$

4.3 Regole di prova: individui particolari e generici

Da $\forall x.P$ possiamo ricavare $P[t/x]$ (**eliminazione dell'universale**); da $P[t/x]$ ricaviamo $\exists x.P$ (**introduzione dell'esistenziale**). Le sostituzioni sono sempre senza cattura. Se tutti gli uomini sono mortali, possiamo istanziare la proprietà su Socrate; sapendo poi che è uomo, il modus ponens conclude che è mortale.

Per *dimostrare* un universale, invece, non basta verificare un individuo noto. Prendiamo un nome nuovo d, senza proprietà particolari, e dimostriamo $P[d/x]$. Se la prova non usa ipotesi su quel nome, il ragionamento vale per ogni individuo: è la **generalizzazione**. Formalmente d non deve comparire nelle ipotesi non scaricate né nella formula che stiamo generalizzando.

Per usare $\exists x.P$ come premessa possiamo dare al testimone un nome nuovo d, assumere temporaneamente $P[d/x]$, e ricavare una conclusione che non nomini d. Il corso chiama questo passo **skolemizzazione**; qui è la regola di uso di un testimone esistenziale. Il nome deve essere nuovo anche rispetto alla conclusione. Non stiamo affermando la legge $(\exists x.P) \Rightarrow P[d/x]$ per una costante prefissata: sarebbe falsa.

Per esempio, da $\forall x.(P(x) \Rightarrow Q(x))$ e $\exists x.P(x)$ vogliamo $\exists x.Q(x)$. Il secondo asserto ci dà un testimone, che chiamiamo d, con $P(d)$. Istanziamo l'universale proprio su quel testimone: $P(d) \Rightarrow Q(d)$. Il modus ponens dà $Q(d)$ e l'introduzione dell'esistenziale dà $\exists x.Q(x)$. La conclusione non conserva il nome temporaneo, quindi possiamo chiudere l'ipotesi sul testimone.

Un **sistema di dimostrazione** organizza queste mosse in regole di inferenza. Una regola deriva una conclusione da premesse; senza premesse è un assioma. Scriviamo $\Gamma \vdash P$ se esiste una sequenza di passi consentiti che deriva P da Γ. Il sistema è **corretto** se ogni risultato derivabile è conseguenza logica; è **completo** se ogni conseguenza logica è derivabile. Per il proposizionale si hanno entrambe le proprietà; per il sistema presentato nel corso usiamo la correttezza. Esistono sistemi completi anche per il primo ordine, ma non una tavola finita che decida in generale ogni formula.

Assumere H per dimostrare Q giustifica l'implicazione $H \Rightarrow Q$: è il contenuto del **teorema di deduzione**, con le condizioni delle regole di quantificazione rispettate. Le ipotesi non diventano leggi. Sotto un quantificatore possiamo sostituire formule equivalenti, o usare le polarità del capitolo 2, soltanto se la relazione usata vale uniformemente per tutti i valori che il quantificatore esamina. Un'ipotesi vera soltanto per l'attuale valore di x non può essere generalizzata senza giustificazione.

4.4 Uguaglianza e asserzioni sugli stati

Con l'interpretazione usuale dell'uguaglianza, $t = u$ è vera quando i termini denotano lo stesso elemento. La **legge di Leibniz** dice che sostituire un individuo con uno uguale non cambia le proprietà:

$$x = y \Rightarrow (P \equiv P[y/x]).$$

La sostituzione riguarda le occorrenze libere e non deve catturare variabili. In particolare, $x = y$ insieme a P permette di concludere $P[y/x]$. Per esempio, $x = y$ ed $x > 0$ danno $y > 0$.

Le **leggi del singoletto** precisano che quantificare su un solo individuo equivale a sostituirlo. Per nomi di variabile distinti x, y, sempre evitando catture:

$$(\forall x.(x = y \Rightarrow P)) \equiv P[y/x],$$

$$(\exists x.(x = y \wedge P)) \equiv P[y/x].$$

Nella seconda formula l'unico possibile testimone è il valore di y: se soddisfa P, l'esistenziale è vero; se non la soddisfa, nessun altro valore può sostituirlo, perché violerebbe $x = y$. La prima formula controlla lo stesso unico individuo mediante un'implicazione.

Siamo pronti a parlare dei programmi. Un'asserzione come $x \geq 0 \wedge y > 0$ è una formula aperta; uno stato fornisce i valori delle variabili di programma, come un assegnamento. Scriviamo $\sigma \vDash P$ se P è vera con quei valori nell'interpretazione numerica usuale. Il prossimo capitolo userà coppie di asserzioni per distinguere ciò che chiediamo prima di un programma da ciò che vogliamo ottenere dopo.

4.4.1 Per proseguire: leggi sui domini e indebolimento

Per verificare una proprietà sull'unione di due gruppi, la verifichiamo su entrambi: $(\forall x{:}\, P \vee Q.R) \equiv ((\forall x{:}\, P.R) \wedge (\forall x{:}\, Q.R))$. Per trovare un testimone nell'unione basta trovarlo in uno dei gruppi: la legge esistenziale scambia $\wedge$ esterna con $\vee$. Anche le leggi distributive già viste e De Morgan valgono con uno stesso filtro ristretto.

L'indebolimento mantiene la direzione consueta: $(\forall x.P) \Rightarrow \forall x.(P \vee Q)$, $(\exists x.P) \Rightarrow \exists x.(P \vee Q)$, e dalle versioni con $P \wedge Q$ possiamo sempre conservare soltanto P. I quantificatori non invertono la polarità: una relazione valida per tutti gli individui può essere trasportata nelle loro portate.

VERIFICA 4

1. Nega $\forall x.(p(x) \Rightarrow \exists y.q(x, y))$, portando la negazione fino agli atomi. Spiega che cosa dovresti trovare per mostrare falsa la formula iniziale.

2. Su $D = \{a, b\}$, la relazione R è vera soltanto per (a, b) e (b, a). Valuta $\forall x.\exists y.R(x, y)$ ed $\exists y.\forall x.R(x, y)$. Mostra poi con un esempio perché da $P(d)$ per una costante già nota non si può sempre generalizzare a $\forall x.P(x)$.

Stati, espressioni e comandi

Per capire un programma dobbiamo seguire due operazioni diverse: leggere un valore e modificare un valore. Nell'espressione `x + 1` leggiamo `x` e calcoliamo un numero. Nel comando `x := x + 1` usiamo quel numero per aggiornare `x`. Il segno `:=` indica un'azione, mentre $=$ nelle asserzioni indica un'uguaglianza.

5.1 Lo stato e la valutazione delle espressioni

Uno stato σ associa valori agli identificatori del programma. Per esempio $\sigma(x)=10$ e $\sigma(y)=3$. Nel linguaggio considerato i valori sono interi o booleani: Val$=\mathbb{Z}\cup$Bool. I due valori booleani sono vero e falso. Lavoriamo con interi matematici, senza un limite di memoria o fenomeni di traboccamento.

Si possono descrivere stati parziali, che non assegnano un valore a tutti i nomi possibili. Nel seguito ogni identificatore letto si assume presente nello stato e del tipo dichiarato. Questa ipotesi sarà importante quando diremo che la lettura di una variabile è sempre definita.

Le espressioni si costruiscono da costanti intere, `true`, `false` e identificatori; poi si combinano con parentesi e operatori. Gli operatori aritmetici sono `+`, `-`, `*`, `div` e `mod`; confronti e operatori logici producono booleani. `true` e `false` sono nomi riservati. Le precedenze usuali distinguono `8 + 5 * 3` da `(8 + 5) * 3`: nel dubbio le parentesi rendono esplicita la struttura.

Prima di eseguire un'espressione controlliamo i tipi. `(x + 5) and (x > y)` può rispettare la grammatica, ma `and` richiede due booleani e `x + 5` è un intero. Il controllo dei tipi è detto **semantica statica**: analizza la forma senza eseguire il programma. Da qui in poi consideriamo espressioni correttamente tipate.

Scriviamo $\mathcal{E}(E,\sigma)$ per il valore dell'espressione E nello stato σ. Per calcolare `(x + 1) * y` nello stato dell'esempio, leggiamo $\sigma(x)=10$, otteniamo $10+1=11$, leggiamo $\sigma(y)=3$ e infine moltiplichiamo: $11\cdot 3=33$. Lo stato non cambia.

La definizione generale segue la costruzione dell'espressione:

$$\mathcal{E}(x,\sigma)=\sigma(x), \qquad \mathcal{E}(E+E',\sigma)=\mathcal{E}(E,\sigma)+\mathcal{E}(E',\sigma).$$

Le costanti denotano i valori corrispondenti; `true` denota vero e `false` falso. Gli altri operatori si interpretano nello stesso modo, applicando l'operazione ai valori dei sottoargomenti; le parentesi non cambiano il valore e `not` nega un booleano. Qui `and` e `or` valutano entrambi gli argomenti: non assumiamo il cortocircuito di alcuni linguaggi reali.

Il controllo dei tipi non impedisce ogni errore. `x div (y - 3)` è ben tipata, ma nello stato con $y=3$ richiede una divisione per zero. Perciò $\mathcal{E}$ è una **funzione parziale**: per alcune coppie espressione-stato il valore non esiste. Lo indichiamo con $\perp$. Per tutti gli interi a, b con $b\neq 0$, fissiamo la convenzione a div $b=\lfloor a/b\rfloor$, il più grande intero non maggiore di a/b, e a mod $b=a-b(a \text{ div } b)$. Con dividendo non negativo e divisore positivo sono il consueto quoziente e il resto non negativo minore del divisore. Gli esempi di divisione del volume usano sempre questo caso; la convenzione sugli altri segni rende precisa la semantica anche fuori dagli esempi.

5.2 Assegnamento, sequenza e perdita di informazione

La scrittura $\sigma[v/x]$ indica lo stato uguale a σ su tutti i nomi tranne x, al quale associa v. Il comando `x := E` prima valuta tutta E nello stato vecchio, poi aggiorna x. Per esempio, da $x=10$, $y=3$, il comando `x := x + y` produce $x=13$, $y=3$.

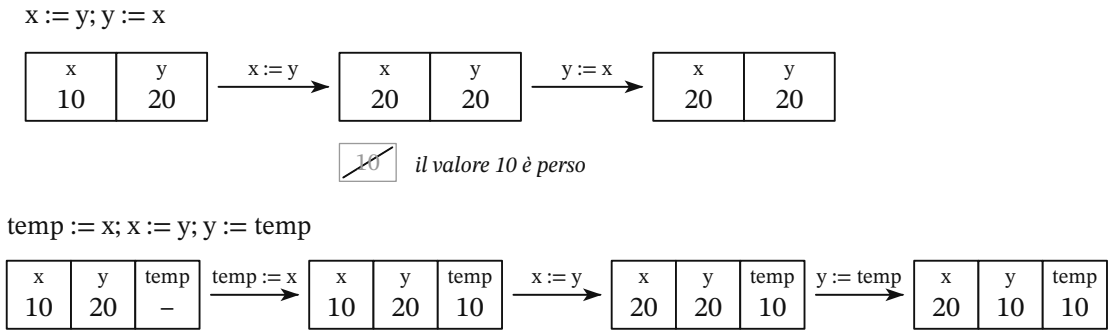
Concetto chiave · Espressioni e comandi
Un'espressione legge lo stato e calcola un valore. Un comando viene eseguito e, se termina con successo, produce uno stato. Un assegnamento sostituisce il vecchio valore della variabile destinazione.

Una sequenza `C; D` esegue prima `C`, poi `D` nello stato raggiunto. Per questo `x := y; y := x` non scambia i valori. Da $x = 10$, $y = 20$, il primo comando porta a $x = 20$, $y = 20$; il secondo legge il nuovo $x = 20$ e lascia ancora $y = 20$. Il vecchio 10 è stato perso.

Per conservarlo lo copiamo prima in una variabile ausiliaria:

```
temp := x;
x := y;
y := temp
```

Dopo il primo comando `temp` contiene 10; dopo il secondo `x` contiene 20; dopo il terzo `y` recupera 10. La necessità dell'appoggio nasce dal fatto che ogni comando legge lo stato corrente, non quello iniziale.



Lo scambio conserva temporaneamente il valore che il secondo assegnamento sovrascriverà. Le frecce seguono la direzione dell'esecuzione.

5.3 Scelte e ripetizioni: if e while

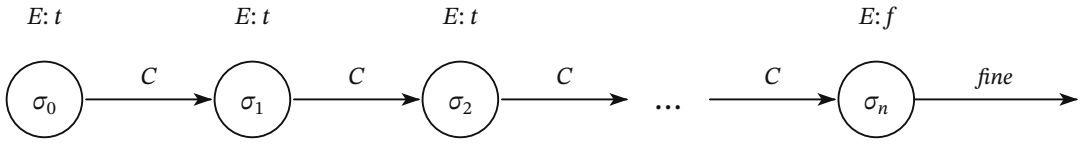
Il comando `skip` non fa nulla e lascia lo stato invariato. Il condizionale `if E then C else D fi` valuta la guardia booleana E. Se è vera esegue soltanto C; se è falsa esegue soltanto D. Se la guardia non è definita, non può scegliere il ramo.

Per ottenere il valore assoluto di `x` in `r` usiamo:

```
if x > 0 then r := x else r := -x fi
```

Con $x = 7$ scegliamo il primo ramo e otteniamo $r = 7$. Con $x = -7$ scegliamo il secondo e otteniamo ancora $r = 7$. Con $x = 0$ la guardia è falsa e il secondo ramo assegna $-0 = 0$. La variabile `x` resta invariata.

Il ciclo `while E do C endw` controlla la guardia prima di ogni eventuale esecuzione del corpo C. Se è falsa, esce subito. Se è vera, esegue il corpo e torna a controllare la guardia nello stato nuovo. Non decide in anticipo il numero di ripetizioni.



Ogni freccia rappresenta un'intera esecuzione del corpo. Il controllo della guardia avviene anche nello stato iniziale; il primo controllo falso conclude il ciclo.

Consideriamo una potenza, con base=2 ed esp=3:

```
ris := 1;
while esp > 0 do
    ris := ris * base;
    esp := esp - 1
endw
```

Al primo controllo abbiamo $(\text{ris}, \text{esp}) = (1, 3)$. Il corpo moltiplica il risultato per 2 e diminuisce l'esponente residuo: i controlli successivi incontrano $(2, 2)$, $(4, 1)$ e $(8, 0)$. Nell'ultimo stato la guardia è falsa, quindi il risultato è 8. Se l'esponente iniziale è zero, il corpo non viene eseguito e rimane 1, come richiede $A^0 = 1$.

La traccia spiega questa esecuzione, ma non prova il risultato per ogni esponente. Per farlo cercheremo una relazione che leghi sempre il prodotto già calcolato al lavoro ancora da svolgere: sarà l'idea di invariante.

5.4 La semantica dei comandi e le esecuzioni senza risultato

Scriviamo $\mathcal{C}(C, \sigma)$ per lo stato finale prodotto da C a partire da σ. Per i primi costrutti:

$$\mathcal{C}(\text{skip}, \sigma) = \sigma,$$

$$\mathcal{C}(\text{x} := \text{E}, \sigma) = \sigma[\mathcal{E}(E, \sigma)\,/\,x],$$

$$\mathcal{C}(C; D, \sigma) = \mathcal{C}(D, \mathcal{C}(C, \sigma)).$$

La formula della sequenza riflette esattamente l'ordine appena visto. Per il condizionale si sceglie $\mathcal{C}(C, \sigma)$ oppure $\mathcal{C}(D, \sigma)$ secondo il valore della guardia. Per $W \equiv$ while E do C endw valgono due clausole: se $\mathcal{E}(E, \sigma)$ è falso, $\mathcal{C}(W, \sigma) = \sigma$; se è vero, $\mathcal{C}(W, \sigma) = \mathcal{C}(W, \mathcal{C}(C, \sigma))$. La seconda clausola rinvia al ciclo stesso: dopo il corpo ricomincia lo stesso controllo.

Anche $\mathcal{C}$ è parziale. Può mancare uno stato finale perché si incontra un errore, come in `x := 1 div 0`, oppure perché l'esecuzione continua per sempre. Nel ciclo `while x >= y do x := x + 1 endw`, se inizialmente $x \geq y$, ogni giro rende x ancora maggiore e la guardia non diventa mai falsa. Se inizialmente $x < y$, invece, il ciclo termina con zero iterazioni. Sono comportamenti diversi dello stesso comando su stati diversi.

Il ciclo con `x := x - 1` al posto dell'incremento termina per ogni coppia di interi: y resta fisso e x diminuisce di uno finché diventa minore di y. La prova generale di questa affermazione richiederà una misura naturale del lavoro residuo, introdotta nel capitolo 7.

Abbiamo usato due punti di vista complementari. La **semantica operazionale** segue i passi dell'esecuzione; la **semantica denotazionale** associa al comando la funzione parziale che trasforma lo stato iniziale in quello finale. Nel capitolo seguente useremo la **semantica assiomatica**: descriveremo il comportamento mediante proprietà prima e dopo i comandi e regole che consentono di provarle.

Verifica 5

1. Esegui `x := x + y; y := x - y; x := x - y` da $x = 3, y = 7$. Poi usa valori iniziali generici A, B per spiegare il comportamento per tutti gli interi.

2. Per `while x ≠ 0 do x := x - 2 endw`, confronta le esecuzioni da $x = 4$ e da $x = 5$. Che cosa mostra l'esempio sulla necessità di controllare la terminazione?

Triple di Hoare e invarianti dei cicli

Una traccia descrive ciò che accade su un dato iniziale. Ora vogliamo una garanzia: da ogni stato ammesso dalla specifica, il programma deve produrre il risultato richiesto. La logica dei capitoli precedenti ci permette di descrivere insiemi di stati; le regole di Hoare collegano queste descrizioni ai comandi.

6.1 Precondizione, postcondizione e correttezza

Un'asserzione è una formula interpretata nello stato del programma. Scriviamo $\sigma \vDash P$ quando P è vera nello stato σ. Con $\sigma(x) = 18$ e $\sigma(z) = -8$, per esempio, $\sigma \vDash x > 0 \land z < 0$, perché l'istanza $18 > 0 \land -8 < 0$ è vera. La notazione P^σ indica proprio questa istanza. La verità dipende solo dalle variabili libere di P: cambiare una variabile che P non legge non può cambiarne la verità. Analogamente il valore di un'espressione dipende solo dagli identificatori che essa legge.

Concetto chiave · Tripla di Hoare
La tripla $\{P\}\ C\ \{Q\}$ ha precondizione P, comando C e postcondizione Q. In **correttezza totale**, da ogni stato che soddisfa P il comando termina con successo in uno stato che soddisfa Q. In **correttezza parziale**, si richiede soltanto che ogni esecuzione che produce uno stato finale soddisfi Q.

La seconda nozione non garantisce che uno stato finale esista. Nella convenzione semantica qui adottata, errori ed esecuzioni infinite non producono uno stato finale: una prova soltanto parziale non esclude neppure gli errori. Manteniamo quindi espliciti i controlli di definitezza quando vogliamo un'esecuzione con successo. Scriveremo $\vDash_{tot}$ oppure $\vDash_{par}$ se occorre distinguere le due letture.

La tripla $\{x > 1\}\ \mathrm{x} := \mathrm{x} + 1\ \{x > 2\}$ è totalmente corretta: il nuovo valore è il precedente più uno, quindi supera 2. La tripla $\{x = 5\}\ \mathrm{x} := \mathrm{x} + 1\ \{x = 5\}$ è falsa: l'unico valore finale possibile di x è 6.

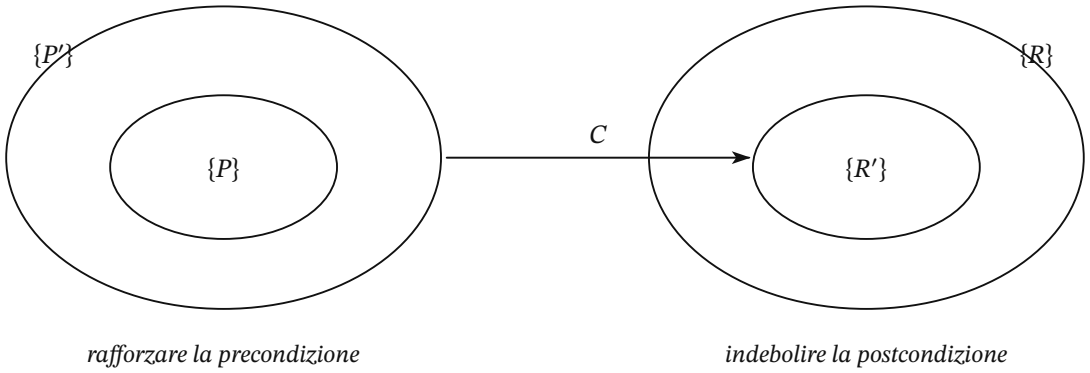
Possiamo usare una tripla per descrivere un comando, cercando Q; per costruire un programma, cercando C; oppure per verificare una proposta, avendo già tutti e tre gli elementi. Una postcondizione va letta nello stato finale. Così $\{x > 0\}\ C\ \{y = 2x\}$ ammette anche `y := 10; x := 5`. Se desideriamo il doppio dell'ingresso, dobbiamo fissarlo con una variabile di specifica: $\{x = A \land A > 0\}\ C\ \{y = 2A\}$. La lettera A resta fissata durante la prova e non compare nei comandi.

6.2 Conseguenza e definitezza delle espressioni

Se un programma funziona su tutti gli stati con $x > 0$, funziona anche su quelli con $x > 10$, che sono meno. È il **rafforzamento della precondizione**. Se garantisce $x = 5$, garantisce anche $x > 0$, che chiede meno: è l'**indebolimento della postcondizione**. La regola di conseguenza riunisce queste osservazioni:

$$\frac{P \Rightarrow P' \quad \{P'\}C\{Q'\} \quad Q' \Rightarrow Q}{\{P\}C\{Q\}}.$$

Le implicazioni sono richieste per tutti gli stati considerati. In termini di insiemi, gli stati di P sono contenuti in quelli di P'; gli stati di Q' sono contenuti in quelli di Q. Non possiamo allargare arbitrariamente gli ingressi o pretendere conclusioni più forti.



Una precondizione più forte restringe i punti di partenza; una postcondizione più debole amplia i risultati ammessi.

Per garantire che un'espressione E abbia valore introduciamo l'asserzione $\mathrm{def}(E)$. Le costanti e le variabili presenti nello stato sono definite. Per somme, prodotti, differenze, confronti e operatori booleani occorre che siano definiti gli argomenti. Per divisione e resto occorre anche il divisore non nullo:

$$\mathrm{def}(E \text{ div } D) \equiv \mathrm{def}(E) \wedge \mathrm{def}(D) \wedge D \neq 0.$$

La stessa clausola vale per `mod`. Per negazione e parentesi basta la definitezza dell'espressione interna. Per esempio, $\mathrm{def}(x \text{ div } (z-y))$ equivale a $z \neq y$: la sottrazione è definita, ma il suo risultato non deve essere zero. Con valutazione senza cortocircuito, `x ≠ 0 and y div x > 1` richiede comunque $x \neq 0$ per essere definita.

6.3 Assegnamento: ragionare a partire dal risultato

Vogliamo garantire $x > y$ dopo `x := x + 2`. Il valore finale di x sarà il vecchio $x+2$. Dobbiamo dunque chiedere, prima, $x+2 > y$. Non stiamo sostituendo il programma con una formula: stiamo descrivendo la condizione che il suo risultato deve soddisfare.

In generale $Q[E/x]$ sostituisce E alle occorrenze libere di x in Q, senza catturare variabili. Rinominiamo prima eventuali variabili quantificate che ostacolano la sostituzione. Se E è definita, vale la proprietà fondamentale:

$$\sigma[\mathcal{E}(E,\sigma)/x] \vDash Q \text{ se e solo se } \sigma \vDash Q[E/x].$$

A sinistra valutiamo Q dopo l'aggiornamento; a destra anticipiamo nella formula il valore che verrà assegnato. Questa proprietà giustifica l'assioma dell'assegnamento:

$$\{\mathrm{def}(E) \wedge Q[E/x]\} \ \ \mathrm{x} := \mathrm{E} \ \ \{Q\}.$$

La definitezza garantisce che il comando produca uno stato; la sostituzione garantisce che quello stato soddisfi Q. Se abbiamo una precondizione data P, basta dimostrare $P \Rightarrow \mathrm{def}(E) \wedge Q[E/x]$ e applicare conseguenza. Per `x := 5` e postcondizione $x = 5$, la richiesta diventa $5 = 5$, sempre vera. Per `x := 1 div y`, anche una postcondizione che non legge x richiede $y \neq 0$ in correttezza totale.

Il comando `skip` ha l'assioma $\{P\}$ skip $\{P\}$: nessun valore cambia, quindi ogni asserzione vera prima resta vera dopo.

6.4 Sequenza: trovare l'asserzione intermedia

Per provare $\{P\} \ \ C; D \ \ \{Q\}$ cerchiamo un'asserzione R tale che $\{P\}C\{R\}$ e $\{R\}D\{Q\}$. Il primo comando consegna al secondo uno stato nelle condizioni che gli servono. È la regola della sequenza.

Riprendiamo lo scambio, ora con $x = N \wedge y = M$ all'ingresso e $x = M \wedge y = N$ all'uscita. Per trovare l'espressione E in `t := E; x := y; y := t`, partiamo dal risultato desiderato. Prima dell'ultimo comando `y := t` serve $x = M \wedge t = N$. Prima di `x := y` serve quindi $y = M \wedge t = N$. Prima di `t := E` serve $y = M \wedge E = N$, oltre alla definitezza.

L'ingresso dice $x = N$: scegliamo dunque $E = x$. Il comando deve leggere il valore disponibile in `x`; non può usare N, che appartiene alla specifica. Abbiamo ricavato il programma e le sue asserzioni intermedie nello stesso ragionamento.

Un esempio più breve è `x := x + 1; y := y - 1`, con risultato $x > y$. Prima dell'ultimo comando serve $x > y - 1$; prima del primo serve $x + 1 > y - 1$. La precondizione $x \geq y - 1$ è sufficiente, perché dà $x + 1 \geq y > y - 1$. La regola non esige che la precondizione data sia esattamente quella calcolata: basta che la implichi.

6.5 Condizionale: utilizzare l'informazione della guardia

Per verificare `if E then C else D fi` con ingresso P e uscita Q controlliamo che $P \Rightarrow \mathrm{def}(E)$. Poi dimostriamo $\{P \wedge E\}\, C\, \{Q\}$ e $\{P \wedge \neg E\}\, D\, \{Q\}$. Ogni ramo riceve l'informazione che ha determinato la scelta.

Per `if x < y then m := y else m := x fi` vogliamo $m = \max(x, y)$. La guardia è sempre definita. Nel primo ramo sappiamo $x < y$, dunque $y = \max(x, y)$: è la richiesta prodotta dall'assegnamento. Nel secondo sappiamo $x \geq y$, dunque $x = \max(x, y)$. Non occorre alcuna ipotesi sul vecchio valore di m, perché entrambi i rami lo sovrascrivono.

Consideriamo ora `if x div y > 0 then m := x else m := y fi`. L'ipotesi $x \geq 0 \wedge y \geq 0$ non basta: ammette $y = 0$. Rafforzandola a $x \geq 0 \wedge y > 0$, la guardia è definita e la divisione intera dà $x \operatorname{div} y > 0$ esattamente quando $x \geq y$. Nel ramo vero scegliamo quindi x, nel ramo falso y: in entrambi i casi assegniamo il massimo. L'ipotesi positiva sul divisore serve sia alla sicurezza sia all'interpretazione del test.

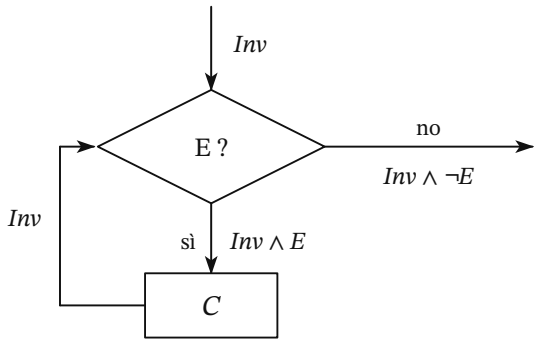
6.6 Invariante e correttezza parziale del while

Passiamo alla divisione mediante sottrazioni, con $x \geq 0$, $y > 0$ e x, y non modificati:

```
q := 0; r := x;
while r >= y do
    q := q + 1;
    r := r - y
endw
```

Da $x = 17, y = 5$ otteniamo ai controlli le coppie $(q, r) = (0,17), (1,12), (2,7), (3,2)$. Che cosa collega queste righe? Ogni volta aumenta di 5 la quantità $q\,y$ e diminuisce di 5 il resto r. Si conserva $x = q\,y + r$. Inoltre $r \geq 0$: il corpo sottrae y solo quando $r \geq y$.

Scegliamo perciò $I \equiv (x = q\,y + r \wedge r \geq 0 \wedge y > 0)$. È un **invariante**: vale dopo l'inizializzazione e viene ripristinato dopo ogni intera esecuzione del corpo. Dopo il solo incremento di q l'uguaglianza può essere falsa; torna vera dopo la sottrazione da r. Il punto in cui la richiediamo è il controllo della guardia.



La proprietà deve valere prima del controllo e dopo l'intero corpo. All'uscita aggiungiamo l'informazione che la guardia è falsa.

La regola generale, in correttezza parziale, è

$$\frac{\{I \wedge E\}\ C\ \{I\}}{\{I\}\ \text{while E do C endw}\ \{I \wedge \neg E\}}.$$

Perché funziona? Prima della prima iterazione vale I. Se vale prima di un giro eseguito, è vera anche E e la premessa ripristina I. Ripetendo questo ragionamento per il numero di giri effettivamente completati, I vale a ogni controllo raggiunto. Se il ciclo produce uno stato finale, in quello stato la guardia è falsa: otteniamo $I \wedge \neg E$. È un ragionamento per induzione sul numero di iterazioni, senza dover conoscere quel numero in anticipo.

Per una specifica generale servono anche ingresso e uscita: $P \Rightarrow I$ e $I \wedge \neg E \Rightarrow Q$. Nella divisione la guardia falsa dà $r < y$; insieme all'invariante otteniamo $x = q\,y + r \wedge 0 \le r < y$. La prova descrive il risultato di ogni esecuzione che arriva all'uscita. Nel prossimo capitolo dimostriamo che quell'uscita viene raggiunta con successo.

Verifica 6

1. Calcola una precondizione che garantisca $y > 10$ dopo `y := x * 2`. Verifica poi se $x \ge 6$ è sufficiente. Motiva con l'assioma, senza limitarti a provare alcuni valori.

2. La precondizione $z = 5 \wedge x = 3$ garantisce $z > 4$ dopo `x := 0; y := z div x` in correttezza totale? Calcola all'indietro il controllo di definitezza e spiega la risposta.

Correttezza totale e funzioni di terminazione

Nella divisione ogni sottrazione riduce il resto. Questo fatto suggerisce perché il ciclo debba finire, ma dobbiamo precisare che cosa diminuisce e perché non può diminuire per sempre. L'invariante descrive la relazione che vogliamo conservare; una funzione di terminazione descrive il progresso verso l'uscita.

7.1 Una quantità naturale che diminuisce

Con $x = 17$, $y = 5$, il resto ai controlli è $17, 12, 7, 2$. È sempre un intero non negativo e diminuisce strettamente a ogni giro eseguito. Una sequenza di questo genere non può essere infinita: partendo da un naturale k, dopo ogni diminuzione intera si perde almeno uno, quindi non si possono compiere più di k diminuzioni senza scendere sotto zero.

Concetto chiave · Funzione di terminazione
Una funzione di terminazione associa agli stati considerati una quantità t intera, definita e non negativa. Se ogni iterazione completata la fa diminuire strettamente, e ogni iterazione termina con successo, il ciclo non può eseguire infinite iterazioni.

La sola diminuzione non basta. La successione di interi $0, -1, -2, \ldots$ diminuisce senza fermarsi; serve il limite inferiore. Neppure essere reale e positivo basta: $1, 1/2, 1/4, \ldots$ è una successione infinita strettamente decrescente. La scelta dei naturali contiene quindi una parte essenziale dell'argomento.

Per confrontare il valore prima e dopo il corpo usiamo una variabile di specifica fresca V. Nella premessa $t = V$ essa memorizza il valore di partenza; nessun comando può modificarla. Il progresso si scrive

$$\{I \wedge E \wedge t = V\}\ C\ \{t < V\}.$$

Non significa che il programma debba conservare t. Significa che la prova può confrontare il nuovo valore di t con il suo valore vecchio, rimasto fissato in V.

7.2 La regola del while in correttezza totale

Una prova totale deve garantire tre fatti: ogni controllo della guardia ha valore; ogni corpo che viene iniziato termina con successo; il numero di iterazioni è finito. Ecco gli obblighi per $\{P\}\ W\ \{Q\}$, dove W è `while E do C endw`:

1. **Ingresso.** $P \Rightarrow I \wedge \mathrm{def}(E)$: prima del primo controllo valgono l'invariante e la definitezza della guardia.

2. **Preservazione.** $\{I \wedge E\}\, C\, \{I \wedge \mathrm{def}(E)\}$ in correttezza totale: il corpo termina con successo e prepara il controllo successivo.

3. **Limite inferiore.** $I \Rightarrow t \geq 0$, con t intera e definita negli stati di I.

4. **Progresso.** $\{I \wedge E \wedge t = V\}\, C\, \{t < V\}$ in correttezza totale, per ogni valore fissato di V.

5. **Uscita.** $I \wedge \neg E \Rightarrow Q$: quando il ciclo si ferma, le informazioni rimaste danno il risultato richiesto.

La preservazione garantisce che il ragionamento possa continuare a ogni controllo. Se la guardia rimanesse sempre vera, otterremmo infinite diminuzioni fra naturali, impossibili. Dunque incontriamo un controllo falso; l'invariante e l'uscita danno Q. Quando un'inizializzazione precede il ciclo, il primo obbligo è una tripla sul comando iniziale, anziché un'implicazione diretta.

7.3 Divisione intera: completare la prova

Riprendiamo `q := 0; r := x` e il corpo `q := q + 1; r := r - y`, con guardia $r \geq y$. L'invariante è $I \equiv (x = q\,y + r \wedge r \geq 0 \wedge y > 0)$ e scegliamo $t = r$. La positività di y compare nell'invariante perché la useremo per dimostrare il progresso a ogni giro.

L'inizializzazione stabilisce $x = 0 \cdot y + x$ e $r = x \geq 0$. La guardia contiene solo un confronto fra variabili e quindi è definita. Per la preservazione indichiamo con q, r i valori prima del corpo e con q', r' quelli dopo. La sequenza produce $q' = q + 1$ e $r' = r - y$. Perciò

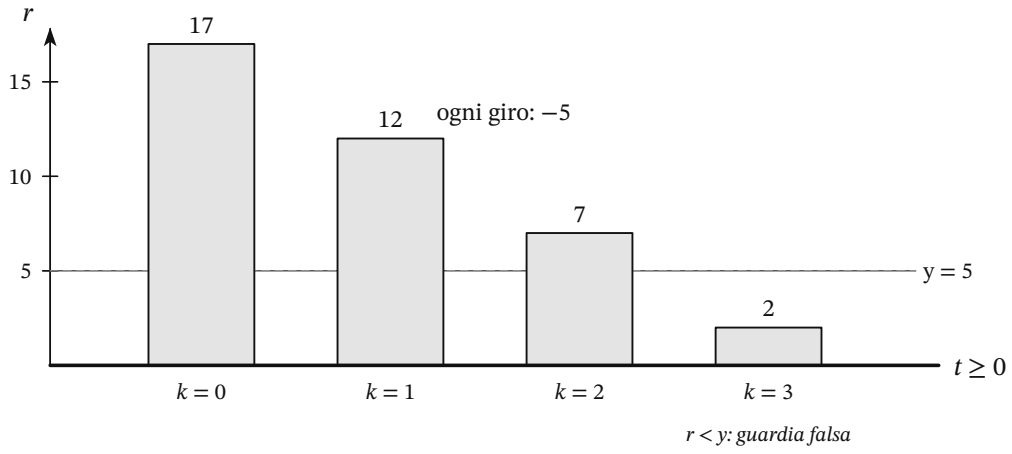
$$q'y + r' = (q+1)y + (r-y) = q\,y + r = x.$$

La guardia $r \geq y$ assicura $r' \geq 0$. Il corpo non assegna a y, quindi $y > 0$ resta vera. Somma e sottrazione sono definite e terminano: la preservazione è totale.

Dall'invariante abbiamo $t = r \geq 0$. Se prima del corpo $r = V$, dopo vale $r' = V - y < V$, perché $y > 0$. Abbiamo così la diminuzione stretta. All'uscita la guardia falsa dà $r < y$; insieme all'invariante otteniamo

$$x = q\,y + r \wedge 0 \leq r < y.$$

Questa è precisamente la caratterizzazione del quoziente e del resto della divisione di x per y. Il programma termina e li calcola per ogni $x \geq 0, y > 0$. Con $y = 0$, invece, la guardia sarebbe sempre vera e la sottrazione non cambierebbe r: fallirebbe il progresso. L'ipotesi sul divisore ha dunque un motivo concreto.



Per $x = 17, y = 5$ il resto scende da 17 a 2. Il limite inferiore e la diminuzione intera spiegano la terminazione; l'invariante spiega il significato del risultato.

7.4 Somma dei primi interi: scegliere l'invariante

Vogliamo sommare gli interi da 0 a $n - 1$, con $n \geq 0$ non modificato. Invece di memorizzare tutti gli addendi, teniamo la somma degli addendi già attraversati e l'indice del prossimo:

```
x := 0; s := 0;
while x < n do
    s := s + x;
    x := x + 1
endw
```

Prima del controllo, x è il prossimo intero da aggiungere. Gli addendi già inclusi sono dunque 0, $1, \ldots, x-1$. Questa lettura suggerisce

$$I \equiv s = \sum_{i=0}^{x-1} i \wedge 0 \leq x \leq n.$$

La somma è vuota e vale zero quando $x = 0$. L'inizializzazione è quindi corretta. I limiti su x servono sia a interpretare la parte già percorsa sia a riconoscere il punto finale.

Per la preservazione chiediamoci che cosa deve valere prima dell'ultimo comando `x := x + 1`. Sostituendo $x+1$ a x nell'invariante otteniamo

$$s = \sum_{i=0}^{x} i \wedge 0 \leq x+1 \leq n.$$

Prima di `s := s + x` serve quindi

$$s + x = \sum_{i=0}^{x} i \wedge 0 \leq x+1 \leq n.$$

La prima uguaglianza viene dall'invariante e dall'identità

$$\sum_{i=0}^{x} i = \sum_{i=0}^{x-1} i + x.$$

La guardia $x < n$, insieme a $0 \leq x$ e all'integralità, dà $0 \leq x+1 \leq n$. Abbiamo verificato entrambe le richieste; tutte le espressioni sono definite.

Per la terminazione prendiamo $t = n - x$, il numero di addendi ancora da attraversare. Da $x \leq n$ segue $t \geq 0$. Un giro aumenta x di uno e lascia n fisso, quindi il nuovo valore è $V - 1 < V$. All'uscita $x \geq n$ e $x \leq n$ danno $x = n$: l'invariante diventa la somma richiesta. Per $n = 0$ non eseguiamo alcun giro e otteniamo correttamente zero.

È essenziale l'ordine degli assegnamenti: aggiungiamo il vecchio x, poi lo incrementiamo. Scambiandoli sommeremmo gli interi da 1 a n, risolvendo un problema diverso.

7.5 Programmi annotati e proprietà non modificate

Annotare un programma significa scrivere asserzioni nei punti utili alla prova. Per la somma possiamo indicare I e $t = n - x$ prima del `while`, $I \wedge x < n$ all'ingresso del corpo, l'asserzione intermedia appena ricavata fra i due assegnamenti e $I \wedge x \geq n$ dopo il ciclo. Queste scritte non vengono eseguite: dichiarano gli obblighi che il lettore deve giustificare.

Spesso serve trasportare un'informazione che il comando non tocca. Sia change(C) l'insieme dei nomi che compaiono come destinazione di un assegnamento in C. Sono nomi potenzialmente modificati, anche se un particolare ramo potrebbe non essere eseguito. Se

$$\text{free}(R) \cap \text{change}(C) = \varnothing,$$

la verità di R non cambia durante un'esecuzione con successo di C. Dalla tripla $\{P\}\,C\,\{Q\}$ possiamo dunque ricavare $\{R \wedge P\}\,C\,\{R \wedge Q\}$. La condizione sulle variabili è ciò che giustifica il passaggio. Nella divisione il corpo assegna solo a q, r, quindi conserva $y > 0$.

Possiamo anche combinare prove dello stesso comando. Da $\{P_1\}C\{Q_1\}$ e $\{P_2\}C\{Q_2\}$ ricaviamo $\{P_1 \wedge P_2\}C\{Q_1 \wedge Q_2\}$: uno stato che soddisfa entrambi gli ingressi gode di entrambe le garanzie sul risultato. Da $\{P\}C\{Q\}$ e $\{R\}C\{Q\}$ ricaviamo $\{P \vee R\}C\{Q\}$: distinguiamo quale ingresso vale e applichiamo la prova corrispondente. In questo linguaggio deterministico la stessa esecuzione finale permette di congiungere le conclusioni.

7.5.1 Per proseguire: eliminare gli incroci

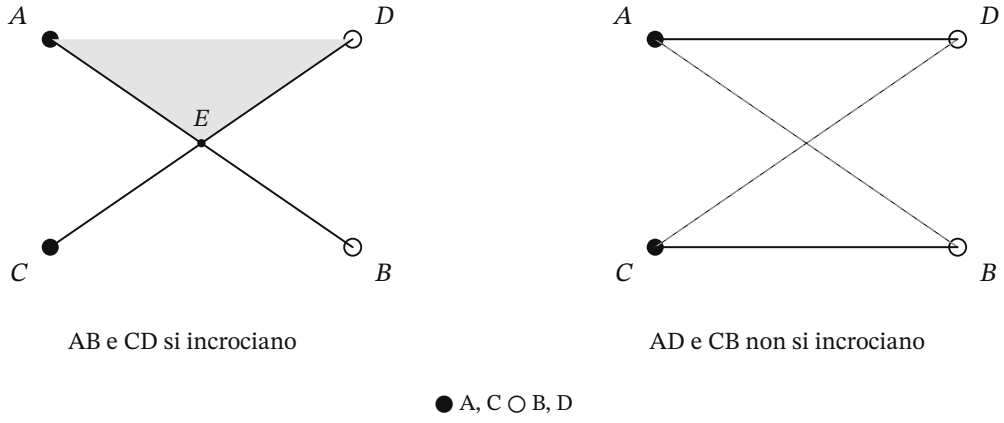
Un esempio geometrico mostra che la funzione di terminazione va scelta pensando al problema, non soltanto al codice. Sono fissati N punti blu e N punti rossi distinti, senza tre punti allineati. Vogliamo abbinarli con segmenti fra colori diversi, senza incroci. Partiamo da un abbinamento qualsiasi; finché due segmenti si incrociano, sostituiamoli con i due collegamenti non incrociati che rispettano i colori.

L'invariante afferma che ogni punto appartiene esattamente a un segmento e ogni segmento unisce colori diversi. Lo scambio lo conserva: ricongiunge gli stessi quattro estremi, ciascuno una sola volta. All'uscita non ci sono incroci, quindi l'obiettivo è raggiunto, se il procedimento termina.

Contare gli incroci non offre direttamente una prova di progresso: togliere l'incrocio scelto può crearne con altri segmenti. Consideriamo invece la somma delle lunghezze. Se $A\,B$ e $C\,D$ si incontrano in E e i colori consentono i nuovi segmenti $A\,D, C\,B$, la disuguaglianza triangolare dà

$$\ell(A\,D) < \ell(A\,E) + \ell(E\,D), \quad \ell(C\,B) < \ell(C\,E) + \ell(E\,B).$$

Le disuguaglianze sono strette: i triangoli coinvolti non sono degeneri, per l'ipotesi di non allineamento. Sommando, la nuova lunghezza totale è strettamente minore di quella vecchia. Gli altri segmenti restano invariati.



I percorsi spezzati che passano per E sono più lunghi dei nuovi collegamenti rettilinei. Il risparmio riguarda la lunghezza totale.

Le lunghezze sono reali, quindi la sola positività non basterebbe. Qui però i punti sono fissati e gli abbinamenti possibili sono finiti: $N!$, perché a ciascun blu associamo un diverso rosso. Anche i valori della lunghezza totale sono finiti in numero. Una diminuzione stretta impedisce di tornare a un valore precedente e quindi non può continuare all'infinito. Questa è una prova completa di terminazione basata sulla finitezza dello spazio degli stati possibili.

Verifica 7

1. Per `while i < n do i := i + 1 endw`, con $I \equiv 0 \le i \le n$ e n fisso, proponi una funzione di terminazione e verifica limite inferiore, progresso e uscita.

2. Nel programma della somma scambia i due assegnamenti del corpo. Trova un invariante che descriva la somma ora calcolata e spiega il risultato finale, compreso il caso $n = 0$.

Costruire un ciclo: il significato delle variabili

Abbiamo imparato a verificare un ciclo quando conosciamo già l'invariante. Ora affrontiamo la domanda che viene prima: come trovare un programma e una proprietà che lo accompagni? Partiamo dal risultato desiderato e cerchiamo informazioni che possano restare vere anche mentre il calcolo è incompleto.

8.1 Dalla postcondizione a una proprietà da conservare

Per ottenere il quoziente della divisione di 17 per 5, possiamo sottrarre ripetutamente 5 e contare le sottrazioni. Dopo due passi abbiamo tolto 10 e rimane 7: l'uguaglianza $17 = 2 \cdot 5 + 7$ racconta dove siamo arrivati. Il resto non è ancora piccolo abbastanza, ma l'uguaglianza è già vera.

Questo spiega la scelta fatta nel capitolo precedente: separare la relazione da conservare, $x = q\,y + r \land r \geq 0$, dalla condizione da raggiungere, $r < y$. La prima diventa un invariante, insieme al vincolo $y > 0$; la seconda suggerisce la guardia opposta, `r >= y`. Introdurre r rende visibile il testimone dell'esistenziale che caratterizza il quoziente: esiste un resto con $x = q\,y + r$ e $0 \leq r < y$.

In generale cerchiamo due proprietà I e G tali che $I \land G$ implichi la postcondizione. Vogliamo conservare I e continuare finché G è falsa. Occorrono poi un'inizializzazione che stabilisca I, un corpo che la ristabilisca e una prova di terminazione. La guardia deve essere un'espressione del linguaggio, quindi non ogni scomposizione matematica è utilizzabile.

> **Concetto chiave · Invariante come significato delle variabili**
> Un buon invariante spiega che cosa rappresentano i valori presenti durante il calcolo. Si rafforza con i vincoli necessari per leggere le espressioni, eseguire il passo successivo e dimostrare la terminazione.

8.2 Fattoriale: ricordare il prodotto già calcolato

Il fattoriale $n!$ è il prodotto da 1 a n; per convenzione $0! = 1$. Supponiamo $n \geq 0$ intero e non modificato. Per $n = 4$ vogliamo ottenere in successione 1, $1 \cdot 1$, $1 \cdot 1 \cdot 2$, $1 \cdot 1 \cdot 2 \cdot 3$, $1 \cdot 1 \cdot 2 \cdot 3 \cdot 4$. Serve una variabile per il prodotto e una per il prossimo fattore.

```
fatt := 1; x := 1;
while x <= n do
    fatt := fatt * x;
    x := x + 1
endw
```

Prima di usare il fattore x, sono già stati moltiplicati gli interi fino a $x - 1$. Cerchiamo dunque

$$I \equiv \text{fatt}{=}(x-1)! \land 1 \leq x \leq n+1.$$

Il limite inferiore 1 assicura che il fattoriale nell'asserzione abbia un argomento naturale. Il limite superiore $n + 1$ include lo stato finale: dopo aver usato n, il contatore avanza ancora una volta.

All'inizio $x = 1$ e fatt=$1 = 0!$; inoltre $1 \le n + 1$. Supponiamo ora che I valga e che $x \le n$. La prima assegnazione produce fatt' $= (x - 1)! \cdot x = x!$. La seconda porta x a $x' = x + 1$. La relazione da ritrovare è fatt' $= (x' - 1)!$, ed è proprio l'uguaglianza appena ottenuta. Anche i limiti si conservano: $1 \le x' \le n + 1$.

La quantità $n + 1 - x$ è un intero non negativo sotto I e diminuisce di 1 a ogni giro. Tutte le espressioni del corpo e della guardia sono definite. Il ciclo termina. All'uscita $x > n$; insieme a $x \le n + 1$ e all'integralità questo dà $x = n + 1$. Dunque fatt=$n!$.

Per $n = 0$ non si entra nel ciclo e si conserva il risultato 1. La funzione $n - x$ scenderebbe invece a -1 nello stato finale: per la regola adottata nel corso, che chiede non negatività sotto tutto l'invariante, scegliamo $n + 1 - x$.

8.3 MCD: trasformare i dati conservando la risposta

Il massimo comune divisore di 18 e 12 è anche il massimo comune divisore di 6 e 12. Infatti un intero divide 18 e 12 se e solo se divide $18 - 12$ e 12: una direzione usa la differenza, l'altra la somma $18 = (18 - 12) + 12$. Sottrarre il minore dal maggiore conserva quindi tutti i divisori comuni.

Con $a, b > 0$ interi e non modificati, lavoriamo su due copie:

```
x := a; y := b;
while x != y do
    if x > y
        then x := x - y
        else y := y - x
    fi
endw
```

Il risultato da conservare non è una somma già accumulata: è il MCD dei due valori di lavoro. Scegliamo

$$I \equiv x > 0 \wedge y > 0 \wedge \mathrm{mcd}(x, y) = \mathrm{mcd}(a, b).$$

L'inizializzazione rende vera questa proprietà. Se $x > y$, allora $x - y > 0$ e la proprietà dei divisori comuni appena dimostrata conserva il MCD. Se si esegue l'altro ramo, sappiamo sia $x \neq y$ sia $\neg(x > y)$: perciò $x < y$, e vale il ragionamento simmetrico. La guardia è importante anche per giustificare questo secondo caso.

Come misurare il lavoro che rimane? Non serve sapere quanti passi occorreranno: basta una quantità che scenda. $x + y$ è positiva sotto I. Nel primo ramo il suo nuovo valore è $(x - y) + y = x$, strettamente minore del vecchio $x + y$; nel secondo è y, ancora minore. Quindi il ciclo termina.

All'uscita $x = y > 0$, e il MCD di due numeri positivi uguali è quel numero. L'invariante dà $x = \mathrm{mcd}(a, b)$. Per esempio, da $(x, y) = (12, 18)$ passiamo a $(12, 6)$ e poi a $(6, 6)$.

8.4 Radice cubica: costruire potenze con differenze

Vogliamo trovare l'intero $r \ge 0$ tale che $r^3 \le x < (r + 1)^3$, per un dato intero $x \ge 0$. Proviamo i cubi nell'ordine $0, 1, 8, 27, \ldots$. Possiamo passare da un cubo al successivo usando soltanto somme? L'identità

$$(b + 1)^3 = b^3 + 3b^2 + 3b + 1$$

dice che ci serve anche il quadrato di b. Conserviamo allora tre informazioni: b è l'indice, $a = b^2$ è il suo quadrato e $y = b^3$ è il suo cubo. Il quadrato si aggiorna grazie a $(b + 1)^2 = b^2 + 2b + 1$.

```
a := 0; b := 0; y := 0;
while y <= x do
    y := y + 3*a + 3*b + 1;
    a := a + 2*b + 1;
    b := b + 1
endw
```

L'ordine salva i valori ancora necessari: `y` deve leggere i vecchi `a` e `b`, e `a` deve leggere il vecchio `b`. Per $x = 10$, le triple (a, b, y) ai controlli della guardia sono $(0, 0, 0)$, $(1, 1, 1)$, $(4, 2, 8)$ e $(9, 3, 27)$. Il ciclo supera il cubo cercato.

Dobbiamo ricordare anche che il cubo precedente era ancora accettabile. Usiamo

$$I \equiv a = b^2 \wedge y = b^3 \wedge (b-1)^3 \le x \wedge b \ge 0 \wedge x \ge 0.$$

L'inizializzazione stabilisce I: in particolare $(-1)^3 = -1 \le x$. Se la guardia è vera, $b^3 = y \le x$. Le due identità mostrano che, dopo il corpo, il nuovo quadrato e il nuovo cubo corrispondono a $b' = b+1$. Inoltre il nuovo cubo precedente è $(b'-1)^3 = b^3 \le x$. Si conserva quindi tutto l'invariante.

Per la terminazione prendiamo $t = x + 1 - (b-1)^3$. Da I segue $t \ge 1$. Dopo un giro $t' = x + 1 - b^3$, e

$$t - t' = b^3 - (b-1)^3 = 3b(b-1) + 1 > 0.$$

Per gli interi $b \ge 0$, il prodotto $b(b-1)$ è non negativo: vale zero per $b = 0, 1$ e poi è positivo. La diminuzione è dunque stretta anche al primo passo.

All'uscita $y > x$, perciò $(b-1)^3 \le x < b^3$. Questo prova che $b-1$ è la parte intera della radice cubica. Il ciclo esegue almeno un giro, poiché all'inizio $y = 0 \le x$, quindi $b \ge 1$ all'uscita e $b - 1 \ge 0$.

> **ATTENZIONE · Quale variabile contiene la radice?**
> Il ciclo lascia in `b` un valore in più. Con $x = 0$ termina con $b = 1$; con $x = 8$ termina con $b = 3$. Per consegnare la radice direttamente in `b` aggiungiamo, dopo `endw`, `b := b - 1`. L'invariante scritto sopra si riferisce al ciclo prima di questa correzione finale.

8.5 Fibonacci: conservare il valore che manca

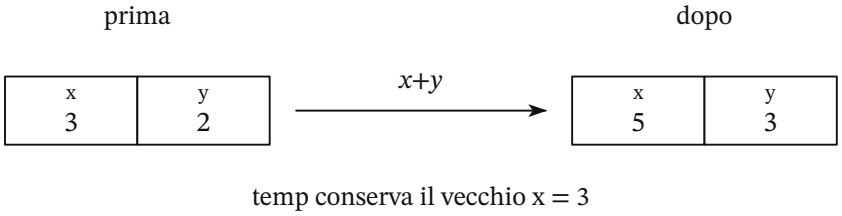
La successione di Fibonacci parte da $\text{fib}(0) = 0$ e $\text{fib}(1) = 1$; ogni termine successivo è la somma dei due precedenti. Per calcolare $\text{fib}(k)$ con $k \ge 1$, vorremmo mantenere $x = \text{fib}(i)$ e far avanzare i fino a k.

Il prossimo valore dovrebbe essere $\text{fib}(i+1) = x + \text{fib}(i-1)$. La formula spiega il passo, ma non è ancora un comando: `fib` non è un'operazione del nostro linguaggio. Conserviamo allora il valore mancante in y. L'invariante diventa

$$I \equiv x = \text{fib}(i) \wedge y = \text{fib}(i-1) \wedge 1 \le i \le k.$$

Per avanzare vogliamo che il nuovo x valga il vecchio $x + y$ e che il nuovo y valga il vecchio x. Con $x = 3, y = 2$, i nuovi valori devono essere 5 e 3. La sequenza `x := x+y; y := x` produrrebbe invece 5 e 5. Salviamo prima il vecchio x:

```
x := 1; i := 1; y := 0;
while i != k do
    temp := x;
    x := x + y;
    y := temp;
    i := i + 1
endw
```



La coppia dei termini consecutivi si sposta di un posto. La variabile temporanea permette di usare il vecchio x anche dopo averlo aggiornato.

L'inizializzazione usa i due valori iniziali della successione. Sotto I e la guardia, $i<k$; la ricorrenza produce $x' = \text{fib}(i+1)$ e il salvataggio produce $y' = \text{fib}(i)$. Dopo l'incremento $i' = i+1$, ritroviamo le stesse relazioni con il nuovo indice, ancora compreso tra 1 e k. La quantità $k-i$ è non negativa e diminuisce di 1. All'uscita $i=k$, quindi $x = \text{fib}(k)$. Per $k=1$ non occorre alcun giro; $k=0$ resta fuori da questa specifica.

8.5.1 Per proseguire: seguire una ricorrenza che dimezza l'indice

Un altro problema del corso mostra che far crescere un contatore non è sempre la scelta migliore. Definiamo $f(0)=0$, $f(1)=1$; per $j \geq 2$ poniamo $f(j)=f(j/2)$ se j è pari e $f(j)=f((j-1)/2)+f((j+1)/2)$ se è dispari. Gli argomenti a destra sono più piccoli di j, quindi la definizione permette di calcolare i valori a partire dai casi iniziali.

Per esempio $f(19)=f(9)+f(10)=f(4)+2f(5)=3f(2)+2f(3)=5f(1)+2f(2)=7$. In ogni passaggio compaiono due indici consecutivi. Cerchiamo allora di conservare

$$I \equiv 0 \leq x \leq n \wedge f(n) = k f(x) + h f(x+1),$$

con $n \geq 0$ non modificato. Inizializzando $x=n, k=1, h=0$ l'uguaglianza è vera. Quando $x=0$ diventa $f(n)=k\cdot 0+h\cdot 1=h$: sappiamo già quale guardia scegliere e dove troveremo il risultato.

Deriviamo il corpo. Scriviamo $q = x \operatorname{div} 2$. Se $x=2q>0$, allora $f(x)=f(q)$ e $f(x+1)=f(q)+f(q+1)$; pertanto

$$k f(x) + h f(x+1) = (k+h) f(q) + h f(q+1).$$

Basta sommare h a k e dimezzare x. Se $x=2q+1$, le identità danno invece

$$k f(x) + h f(x+1) = k f(q) + (k+h) f(q+1).$$

Basta sommare k a h e dimezzare x. Per $x=1$, la prima identità del caso dispari è ancora vera perché $f(1)=f(0)+f(1)$.

Il programma inizializza $x:=n, k:=1, h:=0$. Finché $x \neq 0$, se x è pari esegue `k := h+k`, altrimenti `h := h+k`; in entrambi i casi esegue poi `x := x div 2`. Il test di parità è `x mod 2 = 0`.

Le uguaglianze derivate provano l'invarianza. Sotto la guardia $x>0$, e $0 \leq x \operatorname{div} 2 < x$; dunque x è una funzione di terminazione. Il divisore 2 è sempre non nullo. All'uscita $h=f(n)$, anche per $n=0$, quando il corpo non viene eseguito. Le asserzioni guidano la costruzione; il programma usa solo somme, divisione intera e modulo.

Verifica 8

1. Vogliamo conservare $x\,y + p\,q = n$ mediante `x := x-p; q := E`. Trova un'espressione E che funzioni per tutti gli interi e giustifica anche il caso $p=0$.

2. Nel programma di Fibonacci modifica soltanto l'inizializzazione e l'invariante per calcolare anche il caso $k=0$. Puoi mantenere la stessa forma del corpo usando due termini consecutivi a indici non negativi? Indica quale variabile conterrà il risultato.

Array: indici, aggiornamenti e zone

Finora ogni identificatore di programma ha conservato un intero o un booleano. Per sommare tutti i voti di un elenco, o cambiare ogni elemento di una sequenza, vogliamo invece trattare molti valori con una sola struttura. L'array dà un nome alla sequenza e un indice a ciascuna posizione. Il nuovo compito della verifica è controllare anche quali posizioni leggiamo e modifichiamo.

9.1 Array come funzioni sugli indici

La dichiarazione `v : array [0,4) of int` descrive quattro interi con indici $0, 1, 2, 3$. L'estremo destro 4 segna la fine: non è una posizione leggibile. In generale un array omogeneo, cioè con elementi dello stesso tipo, è una funzione

$$v\colon [a, b) \to A,$$

con indici naturali $a \le b$ e $A = \mathbb{Z}$ oppure $A = \text{Bool}$. L'intervallo $[a, b)$ comprende gli indici $a \le i < b$, ha $b - a$ posizioni ed è vuoto se $a = b$. Scriviamo $\text{dom}(v) = [a, b)$ per il dominio degli indici validi.

Lo stato associa ora al nome v un'intera funzione $\sigma(v)$. Se $\sigma(v) = \{0 \mapsto 2, 1 \mapsto 1, 2 \mapsto 10, 3 \mapsto 6\}$, leggere `v[2]` restituisce 10. Si calcola prima l'espressione indice e poi si applica la funzione:

$$\mathcal{E}(v[E], \sigma) = \sigma(v)(\mathcal{E}(E, \sigma)).$$

Questa formula ha senso solo se il valore dell'indice esiste e appartiene al dominio. Un indice può contenere altre letture: `v[v[0]+v[1]]` legge prima 2 e 1, somma ottenendo 3, poi legge `v[3]`, cioè 6. Dobbiamo controllare sia gli indici interni 0 e 1, sia l'indice esterno 3. Se la somma fosse 4, gli accessi interni sarebbero leciti ma quello esterno fallirebbe.

Concetto chiave · Definitezza di un accesso
Per un'espressione indice intera E,

$$\text{def}(v[E]) \equiv \text{def}(E) \wedge E \in \text{dom}(v).$$

La correttezza richiede che ogni lettura effettivamente eseguita abbia un indice valido. Per un array su $[0, n)$, il solo vincolo $x \le n$ non autorizza a leggere `v[x]`: occorre $0 \le x < n$.

Le cause di indefinitezza comprendono ora divisori nulli e indici fuori dominio. I tipi restano un altro controllo: un array booleano produce booleani, che non possono essere usati come indici interi. Gli operatori del linguaggio agiscono sui singoli elementi: una scrittura come `v+w`, intesa come somma di interi array, non è ammessa.

9.2 Aggiornamento selettivo: prima valutare, poi modificare

Il comando `v[E] := E'` cambia una sola posizione. Tanto l'indice quanto il valore a destra si calcolano nello stato precedente. Per esempio, con l'array appena visto, `v[v[0]] := v[1]+5` calcola l'indice 2 e il valore 6, poi cambia soltanto `v[2]`. Il nuovo array è $\{0 \mapsto 2, 1 \mapsto 1, 2 \mapsto 6, 3 \mapsto 6\}$.

Per descrivere formalmente l'effetto, poniamo $j = \mathcal{E}(E, \sigma)$ e $d = \mathcal{E}(E', \sigma)$. Se entrambi sono definiti e $j \in \text{dom}(v)$, costruiamo la nuova sequenza $w = \sigma(v)[d/j]$: essa vale d all'indice j e coincide con $\sigma(v)$ a ogni altro indice. Infine aggiorniamo lo stato:

$$\mathcal{C}(v[E] := E', \sigma) = \sigma[w/v].$$

L'aggiornamento interno riguarda una posizione della sequenza; quello esterno cambia il valore associato al nome v. Il dominio resta lo stesso.

Per verificare una postcondizione P, scriviamo simbolicamente $w = v[E' / E]$. L'assioma di assegnamento richiede prima del comando

$$\mathrm{def}(E) \wedge \mathrm{def}(E') \wedge E \in \mathrm{dom}(v) \wedge P[w / v].$$

Dopo `v[E] := E'` vale allora P. Come per gli scalari, una precondizione più forte può essere usata tramite la regola della conseguenza. Qui sostituiamo l'intera sequenza nell'asserzione: il nuovo elemento in posizione i è E' se $i = E$, e il vecchio $v[i]$ se $i \neq E$.

Esempio guidato · Estendere un prefisso di somme

Supponiamo $1 \leq k < n$ e che, prima della posizione k, l'array contenga già le somme $a[i] = \sum_{j=0}^{i} j$ per ogni $0 \leq i < k$. Eseguiamo `a[k] := a[k-1]+k`.

I vincoli su k rendono validi entrambi gli indici k e $k-1$. Per dimostrare la nuova proprietà fino a k incluso, separiamo due casi. Se $i < k$, la posizione i non viene toccata e conserva la somma corretta. Se $i = k$, il nuovo valore è

$$a[k-1] + k = \sum_{j=0}^{k-1} j + k = \sum_{j=0}^{k} j.$$

Abbiamo così verificato tutti gli indici $0 \leq i \leq k$. La distinzione tra posizione modificata e posizioni conservate è ciò che permette di provare una postcondizione quantificata.

9.3 Incrementare un array: un confine tra fatto e da fare

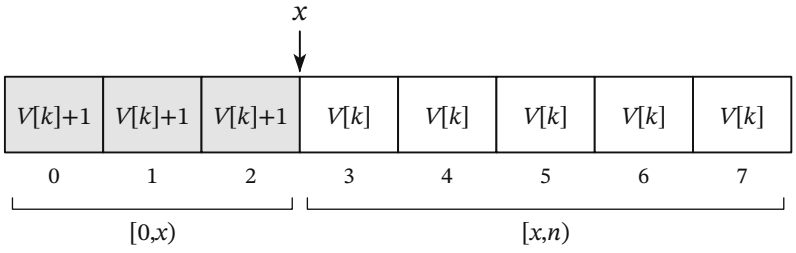
Vogliamo aggiungere 1 a ogni elemento di `a : array [0,n) of int`, con $n \geq 0$. Chiamiamo V la sequenza iniziale, una variabile di specifica che il programma non modifica. Il risultato richiesto è $\forall k \in [0, n).\, a[k] = V[k] + 1$.

Pensiamo a un confine x: a sinistra abbiamo già incrementato, da x in poi dobbiamo ancora farlo. All'inizio $x = 0$; alla fine vogliamo $x = n$. Da questa immagine ricaviamo

$$I \equiv 0 \leq x \leq n \wedge (\forall k \in [0, x).\, a[k] = V[k] + 1)$$

$$\wedge (\forall k \in [x, n).\, a[k] = V[k]).$$

La seconda riga non è superflua: per concludere che incrementare `a[x]` produce $V[x]+1$, dobbiamo sapere che prima vale ancora $V[x]$.



Il confine x separa le posizioni già incrementate da quelle ancora uguali alla sequenza iniziale V. Il confine può anche essere 0 oppure n.

```
x := 0;
while x < n do
    a[x] := a[x] + 1;
    x := x + 1
endw
```

La precondizione è $n \geq 0 \wedge a = V$, dove l'uguaglianza indica che coincidono tutti gli elementi. Con $x = 0$, il prefisso elaborato è vuoto e la proprietà sul resto è esattamente quella iniziale: si stabilisce I.

Prima del corpo, $I \wedge x < n$ dà $0 \leq x < n$: la lettura e la scrittura sono lecite. Le posizioni prima di x restano incrementate, quella in x passa da $V[x]$ a $V[x]+1$, quelle dopo restano iniziali. Abbiamo quindi la stessa divisione con confine $x+1$. Il secondo assegnamento dà a x proprio quel valore, ristabilendo I.

La quantità $n - x$ è non negativa e diminuisce di 1. Il ciclo termina e, quando la guardia è falsa, $x \geq n$; insieme a $x \leq n$ otteniamo $x = n$. L'intero array appartiene allora alla parte elaborata. Per $n = 0$ le proprietà quantificate riguardano l'insieme vuoto e sono soddisfatte senza eseguire il corpo.

9.4 Somma di un array: sostituire la fine con un confine mobile

Ora l'array non deve cambiare; vogliamo soltanto ottenere

$$s = \sum_{i=0}^{n-1} a[i].$$

Sostituiamo nella postcondizione il valore finale n con un indice di lavoro x: s sarà la somma delle prime x posizioni. Per $x = 0$ questa è la somma vuota, che vale 0. Per $x = n$ sarà la somma desiderata.

```
x := 0; s := 0;
while x != n do
    s := s + a[x];
    x := x + 1
endw
```

Usiamo $I \equiv s = \sum_{i=0}^{x-1} a[i] \wedge 0 \leq x \leq n$. L'uguaglianza spiega l'accumulatore; i limiti sul contatore permettono la verifica. Infatti, sotto la guardia $x \neq n$, essi danno $0 \leq x < n$, quindi `a[x]` è definito.

All'inizio I vale per la convenzione sulla somma vuota. Se vale prima del corpo, dopo l'aggiunta otteniamo

$$s' = \sum_{i=0}^{x-1} a[i] + a[x] = \sum_{i=0}^{x} a[i].$$

Poiché il nuovo contatore è $x' = x + 1$, questa è la somma fino a $x' - 1$. Inoltre $x < n$ implica $x' \leq n$, dunque l'invariante è ristabilito. Ancora $n - x$ prova la terminazione; la guardia falsa dà direttamente $x = n$. Con $n = 0$ il risultato è 0.

Sostituire una costante della specifica con una variabile ha trasformato il risultato completo in una proprietà di risultati parziali. I vincoli aggiunti su x servono insieme al controllo degli indici, all'invarianza e alla terminazione.

9.5 Bandiera a due colori: classificare senza perdere elementi

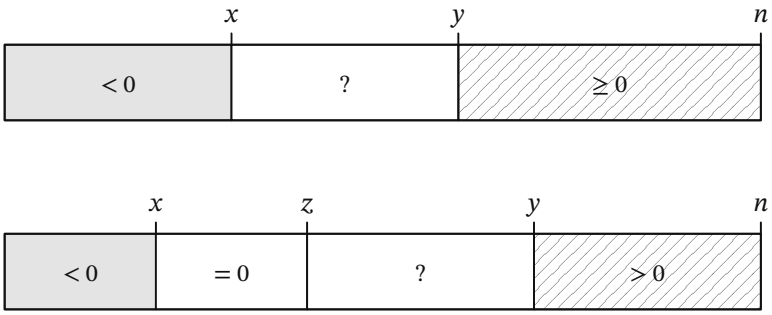
Vogliamo riordinare un array in modo che prima vengano i negativi e poi i valori positivi o nulli. Non chiediamo l'ordinamento interno a ciascun gruppo. Ma chiediamo di conservare tutti i valori, comprese le loro ripetizioni: scriviamo $\mathrm{perm}(a, A)$ se a ha gli stessi elementi della sequenza iniziale A, con le stesse molteplicità.

Un solo confine non basta a distinguere gli elementi già messi a destra da quelli ancora da guardare. Introduciamo due confini $x \le y$: il prefisso $[0, x)$ contiene negativi, il suffisso $[y, n)$ contiene non negativi, mentre $[x, y)$ è ancora sconosciuto. L'invariante è

$$I \equiv \text{perm}(a, A) \wedge 0 \le x \le y \le n$$

$$\wedge (\forall i \in [0, x).\, a[i] < 0) \wedge (\forall i \in [y, n).\, a[i] \ge 0).$$

Non affermiamo nulla sui segni della zona sconosciuta: è proprio il lavoro che resta. La sua lunghezza $y - x$ suggerisce insieme la guardia `x != y` e la funzione di terminazione. Con $x = 0$, $y = n$ entrambe le zone classificate sono vuote.



In alto, due categorie e una zona ancora sconosciuta. In basso, lo stesso ragionamento è esteso a negativi, zeri e positivi: cambiano i confini, mentre resta centrale la zona da esaminare.

Sotto la guardia $x < y$. Possiamo leggere `a[x]` e, se serve, `a[y-1]`: entrambi gli indici sono compresi tra 0 e $n - 1$. Se $a[x] < 0$, abbiamo trovato un altro elemento per il prefisso: basta avanzare x. Se invece $a[x] \ge 0$, vogliamo portarlo a destra, nella posizione $y - 1$, e arretrare y.

Una copia `a[y-1] := a[x]` perderebbe il vecchio elemento di destra. Per conservare la permutazione facciamo uno scambio usando una variabile temporanea. Il valore che arriva in posizione x è ancora sconosciuto: in questo ramo x deve restare fermo.

```
x := 0; y := n;
while x != y do
    if a[x] < 0
        then x := x + 1
        else temp := a[y-1];
             a[y-1] := a[x];
             a[x] := temp;
             y := y - 1
    fi
endw
```

Verifichiamo i due rami. Nel primo l'array non cambia e il solo elemento aggiunto al prefisso è negativo. Nel secondo, lo scambio non modifica né il vecchio prefisso $[0, x)$ né il vecchio suffisso $[y, n)$; mette un non negativo in $y - 1$ e conserva le molteplicità. Dopo l'arretramento, il nuovo suffisso è quindi tutto non negativo. Se $x = y - 1$, lo scambio riguarda una posizione con sé stessa: funziona ugualmente.

Ogni ramo diminuisce $y - x$ di 1, mantenendo $0 \le x \le y \le n$. Il ciclo termina. All'uscita $x = y$, quindi la zona sconosciuta è vuota: il prefisso è negativo e l'intero resto è non negativo. La postcondizione comprende $0 \le x \le n$, con entrambi gli estremi ammessi. Se tutti i valori sono negativi, $x = n$; se nessuno lo è, $x = 0$. Anche l'array vuoto è ammesso.

9.5.1 Per proseguire: bandiera a tre colori

Separiamo ora negativi, zeri e positivi. Per una classificazione completa servono due confini finali; durante il lavoro ne usiamo tre:

$$0 \le x \le z \le y \le n.$$

Manteniamo perm(a, A), negativi in $[0, x)$, zeri in $[x, z)$, positivi in $[y, n)$. La parte sconosciuta è $[z, y)$. Inizializziamo $x = z = 0, y = n$; continuiamo finché $z \neq y$ e misuriamo il lavoro con $y - z$.

Se $a[z] < 0$, dobbiamo portarlo subito dopo il prefisso negativo, cioè in x. Scambiamo $a[x]$ e $a[z]$, poi avanziamo entrambi x e z. Quando $x < z$, il vecchio $a[x]$ era uno zero, quindi può diventare l'ultimo elemento della nuova zona degli zeri. Quando $x = z$, lo scambio è con sé stesso e i confini avanzano insieme senza creare zeri inesistenti.

Se $a[z] = 0$, basta avanzare z. Se $a[z] > 0$, arretriamo y, scambiamo l'elemento positivo con quello della nuova posizione y e lasciamo z fermo: occorre esaminare ciò che è arrivato da destra.

```
x := 0; z := 0; y := n;
while z != y do
    if a[z] < 0
        then temp := a[x]; a[x] := a[z]; a[z] := temp;
             x := x + 1; z := z + 1
        else if a[z] = 0
                then z := z + 1
                else y := y - 1;
                     temp := a[y]; a[y] := a[z]; a[z] := temp
             fi
    fi
endw
```

Sotto la guardia $z < y$, tutti gli accessi sono validi. In particolare, nel ramo positivo il nuovo y è il vecchio $y - 1 \ge z$. Gli scambi conservano le molteplicità; l'analisi dei tre casi mostra che ciascuna zona mantiene il proprio significato. Ogni ramo diminuisce $y - z$ di 1, quindi il ciclo termina. All'uscita $z = y$: la zona sconosciuta scompare e restano, nell'ordine, negativi, zeri e positivi. Questo sviluppo completa l'esercizio proposto nelle lezioni.

VERIFICA 9

1. Per `v : array [0,4) of int` con valori $[2, 1, 10, 6]$, esegui `v[v[0]] := v[0]+v[1]`. Indica tutti gli indici controllati e il nuovo array. Spiega perché il comando successivo `v[v[2]] := 0` è definito oppure non lo è.

2. Modifica il programma della somma per contare gli elementi negativi di un array. Scrivi un invariante che dia significato al contatore, controlla gli indici e giustifica terminazione e risultato anche per l'array vuoto.

Soluzioni ragionate

A.1 Capitolo 1 — Specifiche e stati

1. Prodotto per somme. Gli stati ai controlli sono $(A, B, C) = (4, 2, 0)$, poi $(4, 1, 4)$ e infine $(4, 0, 8)$. C raccoglie la parte già calcolata; B conta le aggiunte rimaste. L'ingresso era $b = 2$, ma B alla fine vale zero. Per questo $A\,B = 0$ nello stato finale non è il prodotto iniziale. La richiesta è $C = a\,b = 8$.

2. Il maggiore. Usiamo x, y per gli ingressi, r per il risultato e a, b per ricordare gli ingressi. All'inizio richiediamo $x = a \land y = b$ con $a, b \in \mathbb{Z}$. Alla fine richiediamo $x = a \land y = b$ e $r \geq a \land r \geq b \land (r = a \lor r = b)$. Le prime due disuguaglianze da sole permetterebbero qualunque numero ancora più grande: l'ultima condizione assicura che il risultato sia uno dei due ingressi. La specifica descrive le proprietà del risultato; non prescrive quale confronto eseguire né come assegnare r.

A.2 Capitolo 2 — Calcolo proposizionale

2.1. «Esco solo se non piove» è $e \Rightarrow \neg p$: uscire richiede che non piova. «Se non piove, esco» è $\neg p \Rightarrow e$: non piovere basterebbe per uscire. Con p falsa ed e falsa la prima formula è vera (non sono uscito), la seconda falsa. Dunque la prima non implica la seconda: posso scegliere di restare a casa anche senza pioggia.

2.2. Distribuiamo per avvicinare P e la sua negazione:

$$P \land (\neg P \lor Q) \equiv (P \land \neg P) \lor (P \land Q) \equiv F \lor (P \land Q) \equiv P \land Q.$$

I passi usano distributività, contraddizione e unità. In $\neg(P \Rightarrow Q) \land R$, l'occorrenza di P attraversa la negazione esterna e l'antecedente: due inversioni, quindi è positiva. Q ne attraversa una sola ed è negativa. R non ne attraversa nessuna ed è positiva. La riscrittura $(P \land \neg Q) \land R$ conferma i tre segni.

A.3 Capitolo 3 — Individui e formule

3.1. In $(\forall x.p(x, y)) \land (\exists y.q(x, y))$ la x dentro p è legata da $\forall x$; la y dentro p è libera. Nella seconda parte x è libera e y è legata. Per sostituire y alla x libera dobbiamo prima cambiare il nome del quantificatore esistenziale, per esempio in z. Otteniamo $(\forall x.p(x, y)) \land (\exists z.q(x, z))$, quindi

$$(\forall x.p(x, y)) \land (\exists z.q(y, z)).$$

La prima parte resta invariata, perché lì non c'è una x libera da sostituire. Le due y rimaste sono libere; il quantificatore $\exists z$ non le cattura.

3.2. Sui naturali, con $s(n) = n + 1$, la prima formula è $\forall x.(\text{pari}(x) \Rightarrow \text{pari}(s(s(x))))$. $s(s(x))$ è un termine; l'intera scrittura è una formula chiusa. «Esattamente uno soddisfa p» si formalizza con $\exists x.(p(x) \land \forall y.(p(y) \Rightarrow y = x))$. Senza la parte universale potremmo avere più testimoni. L'uguaglianza dice che ogni eventuale testimone coincide con quello già scelto, quindi ne garantisce l'unicità.

A.4 Capitolo 4 — Semantica e quantificatori

4.1. Per negare l'universale cerchiamo un controesempio; per negare l'implicazione conserviamo la premessa e neghiamo la conclusione:

$$\neg(\forall x.(p(x) \Rightarrow \exists y.q(x, y))) \equiv \exists x.(p(x) \land \neg(\exists y.q(x, y))).$$

Un secondo De Morgan dà $\exists x.(p(x) \wedge \forall y.\neg q(x,y))$. Dobbiamo quindi trovare un individuo che soddisfi p e per il quale nessun individuo y soddisfi la relazione q. Non basta trovare una sola coppia che fallisca q: per il controesempio scelto devono fallire tutte.

4.2. $\forall x.\exists y.R(x,y)$ è vera: quando $x=a$ scegliamo $y=b$, quando $x=b$ scegliamo $y=a$. $\exists y.\forall x.R(x,y)$ è falsa: $y=a$ fallisce per $x=a$; $y=b$ fallisce per $x=b$. Ogni elemento ha un compagno, ma nessuno è compagno di tutti. Infine, sui naturali scegliamo $P(x)$ come «x è pari» e una costante d che denota 2: $P(d)$ è vera, mentre $\forall x.P(x)$ è falsa per 1. Quel nome aveva una proprietà particolare; non rappresentava un individuo generico.

A.5 Capitolo 5 — Stati, espressioni e comandi

1. Da $(x,y)=(3,7)$ si passa a $(10,7)$, poi a $(10,3)$, infine a $(7,3)$. Con ingressi generici A,B, dopo il primo comando abbiamo $(A+B,B)$; il secondo produce $(A+B,A)$ perché legge il nuovo $x=A+B$; il terzo produce (B,A). Si tratta dunque di uno scambio. Il ragionamento usa interi matematici: in un linguaggio con interi limitati la somma potrebbe eccedere la rappresentazione disponibile.

2. Da 4 otteniamo $4,2,0$: dopo due giri la guardia è falsa. Da 5 otteniamo $5,3,1,-1,-3,\ldots$. Ogni valore resta dispari, perché sottraiamo 2, quindi non può mai essere zero: la guardia resta vera e il ciclo non termina. Ridurre una variabile non garantisce di raggiungere il valore su cui è basato il test di uscita.

A.6 Capitolo 6 — Triple di Hoare e invarianti

1. Sostituendo $2x$ a y nella postcondizione otteniamo $2x>10$. La moltiplicazione è sempre definita. La precondizione richiesta è quindi $x>5$, equivalente a $x\geq 6$ per x intero. L'ipotesi proposta basta per l'assioma dell'assegnamento e garantisce il risultato per tutti gli ingressi ammessi.

2. No. Prima del secondo comando serve la definitezza di `z div x`, dunque $x\neq 0$, oltre a $z>4$. Risalendo attraverso `x := 0` sostituiamo 0 a x: la richiesta diventa $0\neq 0\wedge z>4$, falsa in ogni stato. La precondizione data è soddisfacibile, ma il comando divide sempre per zero. La verità di $z>4$ nello stato d'ingresso non garantisce un'esecuzione con successo.

A.7 Capitolo 7 — Correttezza totale

1. Scegliamo $t=n-i$. Da I segue $i\leq n$, dunque $t\geq 0$. Il corpo incrementa i e lascia n fisso: se prima $n-i=V$, dopo vale $n-(i+1)=V-1<V$. Da $i<n$ e dall'integralità segue $i+1\leq n$, quindi il corpo conserva l'invariante. La guardia è definita e il corpo termina con successo. All'uscita $i\geq n$; insieme a $i\leq n$ ricaviamo $i=n$.

2. Dopo `x := x + 1` si aggiunge il nuovo x. L'invariante adatto ai controlli è $s=\sum_{j=1}^{x} j \wedge 0\leq x\leq n$. All'inizio la somma è vuota e vale zero. Se prima $x=k$, il corpo porta x a $k+1$ e aggiunge proprio $k+1$ a una somma che arriva a k, ripristinando l'invariante. La funzione di terminazione resta $n-x$. All'uscita $x=n$ e il risultato è $s=\sum_{j=1}^{n} j$. Per $n=0$ la somma è vuota e il ciclo restituisce zero senza eseguire il corpo.

A.8 Capitolo 8 — Costruire un ciclo

1. Compensare una modifica. Dopo `x := x-p`, il termine $x\,y$ ha perso $p\,y$. Possiamo compensare aumentando q di y, così il termine $p\,q$ guadagna $p\,y$. Scegliamo dunque $E=q+y$. In termini dei vecchi valori, il risultato è

$$(x-p)y+p(q+y)=x\,y-p\,y+p\,q+p\,y=x\,y+p\,q=n.$$

L'espressione scelta non contiene x, quindi il primo assegnamento non cambia il suo valore. Se $p=0$, il termine $p\,q$ vale sempre zero e la conservazione non impone alcun particolare nuovo valore a q. La scelta $q+y$ funziona comunque ed è definita per tutti gli interi.

2. Fibonacci includendo zero. Mettiamo il risultato nella variabile `y`. Inizializziamo `x := 1; i := 0; y := 0` e conserviamo

$$I \equiv x = \mathrm{fib}(i+1) \wedge y = \mathrm{fib}(i) \wedge 0 \le i \le k.$$

Il corpo resta `temp := x; x := x+y; y := temp; i := i+1`. Se prima i valori sono $\mathrm{fib}(i+1)$ e $\mathrm{fib}(i)$, dopo gli assegnamenti sono $\mathrm{fib}(i+2)$ e $\mathrm{fib}(i+1)$. Con il nuovo indice $i' = i+1$ ritroviamo l'invariante. Sotto `i != k`, i limiti danno $i < k$, quindi l'incremento non supera k. La funzione $k - i$ prova la terminazione; all'uscita $i = k$ e $y = \mathrm{fib}(k)$. Per $k = 0$ la guardia è subito falsa e $y = 0$, come richiesto.

A.9 Capitolo 9 — Array

1. Valutare prima di aggiornare. La lettura `v[0]` nell'indice produce 2. A destra leggiamo gli indici 0 e 1 e otteniamo $2 + 1 = 3$. Tutti gli accessi sono validi perché 0, 1 e 2 appartengono a $[0, 4)$. Cambia soltanto la posizione 2: il nuovo array è $[2, 1, 3, 6]$. Il comando successivo legge `v[2]`, ora uguale a 3, e scrive quindi in posizione 3: è definito e lascia $[2, 1, 3, 0]$. Se avessimo valutato il secondo comando prima del primo, il valore 10 in posizione 2 avrebbe prodotto un indice non valido. L'ordine dell'esecuzione cambia anche la definitezza.

2. Contare i negativi. Sia c il numero di negativi già incontrati. Inizializziamo `x := 0; c := 0`. Finché `x != n`, eseguiamo `if a[x] < 0 then c := c+1 else skip fi`, poi `x := x+1`. Il confronto usa qui il simbolo matematico $<$ per «minore di».

L'invariante è $0 \le x \le n$ insieme a «c è il numero degli indici $i \in [0, x)$ per cui $a[i] < 0$». All'inizio non ci sono indici nel prefisso e il numero è 0. Sotto la guardia, $0 \le x < n$ rende valida la lettura. Se l'elemento è negativo, l'insieme degli indici contati guadagna un elemento, altrimenti non cambia; il ramo scelto aggiorna c proprio di conseguenza. Incrementare x rende il nuovo prefisso uguale al vecchio prefisso più la posizione appena esaminata. La quantità $n - x$ è non negativa e diminuisce di 1. All'uscita $x = n$, dunque il contatore riguarda tutto l'array. Se $n = 0$, vale già $c = 0$ e non si legge alcuna posizione.